

중고 거래 플랫폼

https://github.com/developer-commit/tiny_second_hand_shopping_platfrom

0. 목표 설정 (Goal)

- 목표: 사용자 간 안전하고 원활한 중고 물품 거래가 가능한 C2C 플랫폼 구축
- 핵심 가치:
 - 비트코인 캐시(BCH) 기반의 개인 지갑 및 안전결제(에스크로) 지원
 - 실시간 커뮤니케이션(전체 채팅 및 1:1 개인 채팅) 및 실시간 알림 제공
 - 리뷰 시스템과 자동화된 모더레이션 시스템을 통한 안전한 거래 생태계 조성

1. 요구사항 도출 (Requirements)

1. 회원 관리: 신규 가입, 로그인, 본인인증(2FA), 프로필 관리 및 마이페이지 기능
2. 상품 관리: 상품 등록/수정/삭제, 카테고리 분류, 상품 상태(판매중/예약중/완료) 관리
3. 신뢰 및 소통: 플랫폼 공용/1:1 채팅, 거래 후 평점 및 리뷰 작성
4. 알림 시스템: 채팅, 결제 상태 변경, 신고 결과 등에 대한 실시간 푸시/웹 알림
5. 신고/모더레이션: 불량 유저/상품 신고 및 누적 신고 수 기반 자동 차단/휴면 전환
6. 지갑 및 결제: BCH 지갑 충전/송금/출금, 에스크로 안전결제, 분쟁 중재 및 자동 구매 확정 로직
7. 검색 및 정렬: 키워드 및 태그/카테고리 기반 검색, 정렬 조건 제공
8. 비기능 및 관리자: 데이터 암호화(개인키 보안), 전역 API 권한 제어, 실시간 모니터링 패널

2. 시스템 설계 (System Design)

2.1. 기능 일반화 (System Specifications)

Module 1. 유저 및 계정 관리 (User & Account Management)

- 1.1. 회원가입 및 계정 생성
 - 아이디 중복 확인 및 유일성 검증 기능
 - 스팸 방지를 위한 이메일 또는 SMS 본인 인증 기능 (추가)
- 1.2. 인증 및 보안 (강화)
 - 로그인 / 로그아웃 및 세션 관리
 - 지갑 출금 시 2단계 인증(2FA, OTP 등) 지원 (추가)
- 1.3. 프로필 및 마이페이지
 - 사용자 프로필, 소개글 작성 및 수정 기능
 - 유저 신뢰도 지수(평점) 노출 (추가)

Module 2. 상품 관리 및 카탈로그 (Product Management System)

- 2.1. 상품 등록 및 상태 관리 (강화)
 - 상품 정보(상품명, 가격, 사진, 카테고리, 태그) DB 저장
 - 상품 상태 변경 기능 (판매중 ↔ 예약중 ↔ 판매완료) (추가)
- 2.2. 상품 목록 및 상세 조회

- 목록 화면: 상품명, 가격, 썸네일, **상품 상태**, 조회수 노출
- 상세 화면: 판매자 신뢰도, 전체 정보 노출 및 '채팅하기/안전결제' 분기 처리

Module 3. 검색, 정렬 및 알림 (Search & Notification System)

- 3.1. 상품 검색 기능
 - 제목/태그 검색 및 카테고리별 필터링 (추가)
- 3.2. 알림 기능 (신규)
 - 새로운 채팅 메시지 수신 시 웹/푸시 알림
 - 에스프로 상태 변경(입금, 수령, 정산) 시 시스템 알림

Module 4. 실시간 소통 및 리뷰 (Communication & Review System)

- 4.1. 채팅 시스템
 - 공용 채널 채팅 및 1:1 개인 채팅 기능 (WebSocket 기반)
- 4.2. 평점 및 리뷰 (신규)
 - 에스프로 거래 완료 시 상대방에 대한 리뷰 및 평점(1~5점) 작성 기능
 - 작성된 리뷰에 따른 판매자/구매자 신뢰도 점수 자동 반영

Module 5. 비트코인 캐시(BCH) 지갑 및 결제 (Wallet & Escrow System)

- 5.1. 지갑 및 송금 관리
 - 유저별 BCH 지갑 생성 (보안을 위해 핫지갑/콜드지갑 분리 보관 권장)
 - 출금 시 네트워크 수수료(Network Fee) 계산 및 안내 로직 (추가)
- 5.2. 안전결제(에스프로) 시스템 (강화)
 - 플랫폼 중계 지갑으로 대금 예치 시 **상품 상태 자동 '예약중' 전환**
 - 구매자 수령 승인 시 대금 정산 및 플랫폼 수수료 차감
 - **자동 확정(Timeout):** 구매자가 물품 수령 후 일정 기간(예: 3일) 미응답 시 자동 수령 승인 및 정산 (추가)
 - 수령 거부 시 운영진 중재 및 환불 프로세스 진행

Module 6. 불량 유저 및 콘텐츠 필터링 (Moderation)

- 6.1. 자동 제재 및 상태 전환
 - 특정 신고 횟수 초과 누적 시 상품 차단 / 유저 계정 휴면 전환
 - 악의적 신고 방지를 위해 1유저 1상품 1회 신고 제한 (추가)

Module 7. 시스템 관리자 (Admin & System Operations)

- 7.1. 어드민 패널 및 모니터링
 - 최고 관리자 API 전역 호출 권한
 - DB 실시간 현황, 누적 플랫폼 수수료 수익 조회 (추가)
 - 분쟁 건에 대한 강제 정산/환불 트리거 기능

2.3. 웹페이지 설계 (Web Page Information Architecture)

페이지명	주요 기능 및 화면 구성 요소	연결 URL / 경로
메인 / 목록	카테고리/제목/태그 검색, 정렬 탭, 상품 카드(상품명, 가격, 썸네일, 상품 상태, 조회수 노출), 공용채팅 버튼	/
회원가입	기본 정보 입력, 아이디 중복 확인 및 유일성 검증, 이메일/SMS 본인 인증	/signup
로그인 / 로그아웃	아이디/비밀번호 인증 및 세션 관리 (로그아웃은 공통 헤더/메뉴에 포함)	/login, /logout
마이페이지	내 프로필, 나의 평점/리뷰(신뢰도 지수), 계정 보안(비밀번호, 2FA 설정), 상품/거래 내역 관리	/mypage
지갑 대시보드	BCH 잔액 조회, 입금용 지갑 주소 발급/확인, 전체 입출금 트랜잭션 내역	/wallet
지갑 출금	외부 지갑 주소 입력, 네트워크 수수료(Network Fee) 계산 및 안내, 출금 전 2FA(OTP) 인증 모달	/wallet/withdraw
상품 등록	상품명, 카테고리, 가격, 이미지, 태그 입력 DB 저장 품	/products/new
상품 수정	기존 등록 상품 정보 수정 품, 상품 상태(판매중/예약중/판매완료) 변경 버튼	/products/:id/edit
상품 상세	상세 정보, 판매자 신뢰도, 1:1 채팅/안전결제 버튼, 상품 상태 배지, 신고하기 품(1유저 1상품 1회 제한)	/products/:id
실시간 채팅	공용 채팅 탭, 1:1 채팅 목록, 거래 연동(채팅방 내 송금/결제 버튼)	/chat
안전결제	대금 예치 상태, 수령 승인, 수령 거부 품, 자동 확정(Timeout) 기한 안내, 거래 완료 후 상대방 리뷰/평점 작성 품	/escrow/:trade_id
알림 내역	시스템 알림(에스크로 상태 변경), 채팅 메시지 도착 알림 내역 모아보기	/notifications
관리자 어드민	DB 조회, 자동 제재 현황, 분쟁 중재(강제 정산/환불 트리거) 패널, 플랫폼 지갑 모니터링, 수수료 수익 통계	/admin

2.4 데이터베이스 스키마

```
-- =====
-- Module 1. 유저 및 계정 관리 (Users & Accounts)
-- =====

CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,      -- 아이디 (중복 확인 및 유일성)
  password_hash VARCHAR(255) NOT NULL,
  email VARCHAR(100) UNIQUE,                  -- 이메일 인증용
  phone VARCHAR(20) UNIQUE,                   -- SMS 인증용
  is_verified BOOLEAN DEFAULT FALSE,          -- 본인 인증 여부
  bio TEXT,                                   -- 소개글
  trust_score DECIMAL(3, 2) DEFAULT 0.00,    -- 유저 신뢰도 지수 (평점)
  is_2fa_enabled BOOLEAN DEFAULT FALSE,      -- 2단계 인증(OTP) 활성화 여부
  two_factor_secret VARCHAR(255),            -- 2FA 시크릿 키
```

```

    role VARCHAR(20) DEFAULT 'user',          -- 권한 (user, admin)
    status VARCHAR(20) DEFAULT 'active',      -- 계정 상태 (active, dormant,
suspended)
    reported_count INT DEFAULT 0,             -- 누적 신고당한 횟수
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Module 5. 지갑 관리 (Wallets & Transactions)
-- =====

CREATE TABLE wallets (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT UNIQUE NOT NULL REFERENCES users(id),
    bch_address VARCHAR(100) UNIQUE NOT NULL, -- 입금용 BCH 지갑 주소
    balance DECIMAL(18, 8) DEFAULT 0.00000000, -- BCH 잔액
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE wallet_transactions (
    id BIGSERIAL PRIMARY KEY,
    wallet_id BIGINT NOT NULL REFERENCES wallets(id),
    tx_type VARCHAR(20) NOT NULL,             -- 타입 (deposit, withdrawal)
    amount DECIMAL(18, 8) NOT NULL,          -- 송금액
    network_fee DECIMAL(18, 8) DEFAULT 0,    -- 출금 시 네트워크 수수료
    tx_hash VARCHAR(255) UNIQUE,             -- 블록체인 트랜잭션 해시
    status VARCHAR(20) DEFAULT 'pending',    -- 상태 (pending, confirmed,
failed)
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Module 2 & 3. 상품 관리 및 카탈로그 (Products)
-- =====

CREATE TABLE products (
    id BIGSERIAL PRIMARY KEY,
    seller_id BIGINT NOT NULL REFERENCES users(id),
    title VARCHAR(255) NOT NULL,             -- 상품명
    description TEXT NOT NULL,               -- 상품 상세 설명
    price DECIMAL(18, 8) NOT NULL,          -- 가격 (BCH)
    category VARCHAR(50) NOT NULL,          -- 카테고리
    status VARCHAR(20) DEFAULT 'on_sale',    -- 상태 (on_sale, reserved,
sold)
    view_count INT DEFAULT 0,               -- 조회수
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE product_images (
    id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    image_url VARCHAR(255) NOT NULL,        -- S3 등 이미지 저장 경로
    is_thumbnail BOOLEAN DEFAULT FALSE,     -- 썸네일 여부

```

```

        created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE product_tags (
    id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    tag_name VARCHAR(50) NOT NULL          -- 태그 검색용
);
CREATE INDEX idx_product_tags_name ON product_tags(tag_name);

-- =====
-- Module 5. 안전결제 / 에스크로 (Escrow Trades)
-- =====
CREATE TABLE escrow_trades (
    id BIGSERIAL PRIMARY KEY,
    product_id BIGINT NOT NULL REFERENCES products(id),
    buyer_id BIGINT NOT NULL REFERENCES users(id),
    seller_id BIGINT NOT NULL REFERENCES users(id),
    amount DECIMAL(18, 8) NOT NULL,          -- 예치된 대금
    platform_fee DECIMAL(18, 8) NOT NULL,    -- 플랫폼 수수료
    status VARCHAR(20) DEFAULT 'deposited',  -- 상태 (deposited, received,
disputed, settled, refunded)
    auto_confirm_at TIMESTAMP WITH TIME ZONE, -- 자동 확정(Timeout) 기준 일시
(예: 예치 후 +3일)
    dispute_reason TEXT,                    -- 수령 거부(분쟁) 사유
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Module 4. 평점 및 리뷰 (Reviews)
-- =====
CREATE TABLE reviews (
    id BIGSERIAL PRIMARY KEY,
    trade_id BIGINT UNIQUE NOT NULL REFERENCES escrow_trades(id), -- 1거래 1
리뷰
    reviewer_id BIGINT NOT NULL REFERENCES users(id),
    reviewee_id BIGINT NOT NULL REFERENCES users(id),
    rating INT CHECK (rating >= 1 AND rating <= 5), -- 1~5점
    comment TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Module 4. 채팅 시스템 (Chat System)
-- =====
CREATE TABLE chat_rooms (
    id BIGSERIAL PRIMARY KEY,
    product_id BIGINT REFERENCES products(id) ON DELETE SET NULL, -- 1:1 채
팅 시 연동된 상품
    room_type VARCHAR(20) DEFAULT 'private', -- (public, private)
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE chat_participants (
  room_id BIGINT NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
  user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  joined_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (room_id, user_id)
);

CREATE TABLE chat_messages (
  id BIGSERIAL PRIMARY KEY,
  room_id BIGINT NOT NULL REFERENCES chat_rooms(id) ON DELETE CASCADE,
  sender_id BIGINT NOT NULL REFERENCES users(id),
  message TEXT NOT NULL,
  is_read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Module 6. 불량 유저 신고 (Reports)
-- =====
CREATE TABLE reports (
  id BIGSERIAL PRIMARY KEY,
  reporter_id BIGINT NOT NULL REFERENCES users(id),
  product_id BIGINT NOT NULL REFERENCES products(id),
  reason TEXT NOT NULL,
  status VARCHAR(20) DEFAULT 'pending', -- (pending, reviewed,
action_taken)
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  -- [핵심 로직] 악의적 신고 방지: 1유저 1상품 1회 신고 제한
  UNIQUE (reporter_id, product_id)
);

-- =====
-- Module 3. 알림 시스템 (Notifications)
-- =====
CREATE TABLE notifications (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type VARCHAR(30) NOT NULL, -- (chat, escrow_update,
system, warning)
  reference_id BIGINT, -- 관련된 엔티티의 ID (trade_id,
message_id 등)
  message TEXT NOT NULL,
  is_read BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- Module 7. 관리자 및 플랫폼 통계 (Platform Stats)
-- =====
CREATE TABLE platform_stats (
  id SERIAL PRIMARY KEY,
  total_fee_collected DECIMAL(18, 8) DEFAULT 0.00000000, -- 누적 플랫폼 수수
료 수익

```

```
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

2.5 open api 문서 작성

open.yaml 문서 참고

2.6 구현되어야 하는 보안 요소

1. 계정 및 인증/인가 보안 (Authentication & Authorization)

- 비밀번호 단방향 암호화 저장 (password_hash: Argon2, Bcrypt 등 안전한 해시 알고리즘 적용)
- 신규 가입 시 스팸 및 위장 가입 방지를 위한 이메일/SMS 본인 인증
- 세션 및 인증 토큰(JWT 등) 만료 및 무결성 관리
- 2단계 인증 (2FA / OTP) 지원 및 지갑 출금/보안 설정 시 필수 검증
- 2FA 시크릿 키(two_factor_secret) 양방향 암호화 보관
- 역할 기반 접근 제어 (RBAC: 'user', 'admin' 권한 분리 및 엔드포인트 검증)
- 계정 상태(status: active, dormant, suspended)에 따른 API 접근 제한

2. 가상자산 및 지갑 보안 (Wallet & Crypto Security)

- 핫지갑(Hot Wallet)과 콜드지갑(Cold Wallet)의 분리 보관 및 운영
- 개인키(Private Key) 및 지갑 관련 민감 데이터의 KMS/HSM 기반 암호화 보관
- BCH 출금 요청 시 2FA(OTP) 추가 인증 필수화
- 블록체인 트랜잭션 해시(tx_hash) 검증을 통한 무결성 확보 및 이중 지불 방지

3. 거래 및 에스스로 보안 (Transaction & Escrow Security)

- 제3자 에스스로(안전결제) 예치 시스템을 통한 구매자/판매자 대금 이동 보호
- 자동 구매 확정(Timeout) 및 상태 변경 로직의 변조 방지 처리
- 분쟁 발생 시 운영진 강제 정산/환불 기능에 대한 접근 권한 제한 및 감사 로그 기록

4. 모더레이션 및 어뷰징 방지 (Anti-Abuse & Moderation)

- 신고 누적에 따른 불량 유저 계정 휴면 전환 및 상품 자동 차단 로직
- 악의적/중복 신고 방지 (1유저 1상품 1회 신고 DB UNIQUE 제약조건 적용)
- 유저 신뢰도 지수(trust_score) 자동 계산을 통한 신뢰 생태계 유지

5. 시스템 및 API 보안 (System & API Security)

- 관리자(Admin) 전용 API 전역 접근 제어 (Global Authorization Middleware)
- 개인정보 및 민감 데이터 DB 저장 시 데이터 암호화 (Data-at-Rest Protection)
- API 요청에 대한 Rate Limiting 및 스팸/DDoS 방지

3. 기술 스택 결정

기술 스택 선정 배경

프로젝트를 All Vibe로 진행함에 따라, 코드 검증 단계에서 컴파일러의 강한 타입 체크와 안전성 보장을 활용하고자 프론트엔드(Frontend)와 백엔드(Backend) 모두 **Rust**를 전면 도입하기로 결정하였습니다.

프로젝트의 [Cargo.toml](#)에 직접 등록하여 활용하는 분야별 최적화 라이브러리 목록입니다.

1. 프론트엔드 (Frontend - Leptos)

라이브러리	설명	저장소 링크
leptos	Leptos 코어 (반응형 상태 관리 및 UI 컴포넌트)	GitHub
leptos_router	클라이언트 사이드 SPA 라우터	GitHub
leptos_meta	HTML Head, Title, Meta 태그 관리	GitHub

2. 백엔드 코어 & 웹 프레임워크 (Backend Core)

라이브러리	설명	저장소 링크
tokio	비동기 런타임	GitHub
axum	웹 프레임워크 및 WebSocket 지원	GitHub
tower-http	CORS, Rate Limiting, 로컬 정적파일 서빙(ServeDir)	GitHub

3. 데이터베이스 & ORM (Database)

라이브러리	설명	저장소 링크
sea-orm	비동기 PostgreSQL ORM	GitHub

4. 인증, 권한 및 보안 (Auth & Security)

라이브러리	설명	저장소 링크
argon2	비밀번호 단방향 해싱	GitHub
jsonwebtoken	JWT 세션/인증 토큰	GitHub
totp-rs	2FA / OTP 검증	GitHub
aes-gcm	2FA 시크릿 키 & 지갑 개인키 DB 암호화	GitHub
validator	이메일/폼 데이터 유효성 검증	GitHub

5. 가상자산 지갑 및 외부 통신 (BCH Light Client)

라이브러리	설명	저장소 링크
bip39	HD 지갑 니모닉(시드 문구) 생성 및 복구	GitHub
rust-bitcoin	오프라인 트랜잭션 생성 및 키 파싱	GitHub
cashaddr	BCH 전용 CashAddr 주소 변환 (bitcoincash:q...)	GitHub
reqwest	퍼블릭 API / Electrum HTTP 게이트웨이 통신	GitHub

6. 실시간 메시징 & 백그라운드 스케줄링 (Real-time & Worker)

라이브러리	설명	저장소 링크
redis	채팅/알림 Pub/Sub 메시징 (로컬 Redis)	GitHub
tokio-cron-scheduler	3일 타임아웃 자동 확정 크론 작업	GitHub

7. 로컬 미디어 파일 처리 (Local Storage)

라이브러리	설명	저장소 링크
image	업로드 이미지 썸네일 리사이징 및 로컬 저장	GitHub

8. 공통 유틸리티 (Utilities)

라이브러리	설명	저장소 링크
serde	직렬화 / 역직렬화 (프론트/백엔드 공통)	GitHub
serde_json	JSON 파싱	GitHub
chrono	날짜 및 타임스탬프 처리	GitHub
dotenvy	<code>.env</code> 환경 변수 로드	GitHub
tracing	로깅 및 모니터링	GitHub
thiserror	API 에러 커스텀 정의	GitHub

4. 모델 구조 데모

```
├── Cargo.lock
├── Cargo.toml
├── crates
│   ├── backend
│   │   ├── Cargo.toml
│   │   └── src
│   │       ├── db
│   │       │   ├── chat_message.rs
│   │       │   ├── escrow_trade.rs
│   │       │   ├── mod.rs
│   │       │   ├── platform_stats.rs
│   │       │   ├── product.rs
│   │       │   └── wallet_transaction.rs
│   │       ├── main.rs
│   │       └── utils
│   │           ├── mod.rs
│   │           └── security.rs
│   └── frontend
│       ├── Cargo.toml
│       └── src
```

```

├── main.rs
├── models
│   ├── escrow_model.rs
│   ├── mod.rs
│   ├── notification_store.rs
│   └── product_model.rs
├── shared
│   ├── Cargo.toml
│   └── src
│       ├── dto
│       │   ├── admin_dto.rs
│       │   ├── chat_dto.rs
│       │   ├── escrow_dto.rs
│       │   ├── mod.rs
│       │   ├── noti_dto.rs
│       │   ├── product_dto.rs
│       │   ├── report_dto.rs
│       │   └── transaction_dto.rs
│       ├── dto.rs
│       └── lib.rs
├── openapi.yaml
└── plan.md

```

3대 보안 원칙(식별자 난독화, 스키마 은닉, 관심사 분리)을 적용하여 모델 구조, dto 구조를 결정한다.

5. 프로젝트 구조 구체화

Role

당신은 최고 수준의 Rust 풀스택 아키텍처 설계자이자 보안 전문가입니다.

당신의 목표는 주어진 프로젝트 구조와 라이브러리 스택, 핵심 보안 규격을 바탕으로 비트코인 캐시 (BCH) 기반 중고 거래 플랫폼의 ****완벽한 시스템 뼈대(Scaffold)와 아키텍처 구조화****를 수행하는 것입니다.

openapi.yaml, 프로젝트의 구조의 논리적 모순을 발견시, 수정사항을 plan.md에 추가후 수정해도 무방합니다.

주의:** 실제 세부 비즈니스 로직(내부 작동 코드)을 구현하는 것이 아닙니다.* 데이터 구조(Struct), API 인터페이스(Trait/Handler), 프론트엔드 라우팅 및 상태 래퍼의 시그니처를 설계하여 *견고한 구조(Blueprint)를 만드는 것****이 핵심입니다.

Context

- 당신은 현재 프로젝트의 디렉토리 구조(Workspace: frontend, backend, shared)를 읽고 파악할 수 있습니다.
- 백엔드는 `Axum`과 `SeaORM`, 프론트엔드는 `Leptos`, 통신 규격은 `shared` 크레이트의 DTO로 철저히 분리되어 있습니다.
- plan.md에는 프로젝트의 목표가 담겨 있습니다.
- openapi.yaml에는 api 계약이 적혀있습니다.

 구조적 보안 및 아키텍처 원칙 (뼈대에 반드시 반영)

코드를 구조화할 때 아래 원칙이 ****타입 시스템과 모듈 구조****에 명확히 드러나야 합니다.

1. ****식별자 난독화 (Opaque IDs)****

- ``shared`` DTO 구조체에는 순차 ID(``i64``)가 존재해선 안 됩니다. 반드시 ``String`` (또는 ``OpaqueId`` 같은 커스텀 타입)으로 정의하세요.
- ``backend/src/utis/security.rs``에 난독화 유틸리티의 함수 시그니처(``pub fn obfuscate``, ``pub fn deobfuscate``)와 커스텀 에러(``thiserror``) 구조만 정의하세요.

2. ****스키마 은닉 (Schema Hiding) 및 관심사 분리****

- ****Shared****: ``validator`` 매크로가 적용된 DTO와 직렬화(``serde``) 구조체만 정의. (DB 컬럼명과 다르게 네이밍)
- ****Backend (Entity)****: ``sea-orm`` 모델 구조체와 ``TryFrom<Model> for DTO`` 변환 트레이트의 시그니처 작성.
- ****Frontend (Model)****: ``shared`` DTO를 받아 ``leptos``의 ``RwSignal``이나 ``Resource``로 감싸는 프론트엔드용 상태 구조체(Wrapper) 설계.

3. ****인증, 권한, 지갑 보안의 타입화****

- 비밀번호, 2FA 시크릿 키, 지갑 개인키는 일반 ``String``이 아닌 보안 래퍼 타입(예: ``SecretString``, ``EncryptedKey``)으로 정의하여 메모리 유출을 방지하는 구조를 보여주세요.
- 글로벌 미들웨어(Auth/RBAC)가 적용될 Axum 라우터의 뼈대(Router builder)를 설계하세요.

4. ****디자인 패턴 적용 (외부 의존성 격리 및 비즈니스 로직 캡슐화)****

- 비즈니스 로직(Service)과 인프라스트럭처(DB, 네트워크)를 철저히 분리하세요.
- 특히 지갑(Wallet), 외부 BCH 네트워크 통신, 결제 모듈 등은 외부 환경 변화에 대비하여 ****어댑터 패턴(Ports & Adapters / Hexagonal)****을 적용해야 합니다. Rust의 ``trait`` (Port)으로 인터페이스를 먼저 정의하고, 이를 구현하는 ``struct`` (Adapter)를 인프라 계층에 배치하세요.

🚀 작업 지시 (Scaffolding Execution)

내부 로직은 ``todo!`` ("구체적인 로직 설명") 또는 ``unimplemented!``로 비워두고, ****인터페이스와 구조****에 집중하여 코드를 생성합니다. 사용자가 지정하는 ****[Target Domain]****에 대해 아래 5계층의 뼈대를 작성하세요.

[출력해야 할 구조화 영역]

1. ****[Shared] API Contract (DTOs & Enums)****:

- 프론트와 백엔드가 통신할 Request/Response 데이터 구조. 유효성 검증(``validator``) 어노테이션 포함.

2. ****[Backend] Controller Layer (Axum Handlers & Router)****:

- API 엔드포인트 함수 시그니처, 의존성 주입(State), 사용될 미들웨어(Auth) 구조. (HTTP I/O만 담당)

3. ****[Backend] Service & Domain Layer (Business Logic & Ports)****:

- 비즈니스 핵심 로직 시그니처.
- 외부 인프라(DB, 지갑 통신)를 추상화한 의존성 주입용 ``trait`` (Port) 정의.

4. ****[Backend] Infrastructure Layer (SeaORM Model & Adapters)****:

- DB 스키마 구조체 및 DTO 변환 시그니처 (``TryFrom``/``Into``).
- Service 계층에서 정의한 ``trait``을 구현하여 실제 외부 API(지갑/BCH 네트워크)와 통신하는 어댑터(Adapter) 구조체.

5. ****[Frontend] View & State Layer (Leptos Models & Router)****:

- 상태 관리를 위한 래퍼 구조체(Signal/Resource)와 페이지 컴포넌트(``#[component]``) 뼈대.

📝 코드 출력 규칙

- 각 파일 및 코드 블록 상단에 주석으로 `**"이 구조가 어떤 아키텍처/보안 목적을 달성하는지"*` 간략히 설명하세요.
- 세부 비즈니스 로직 작성에 토큰을 낭비하지 마세요. 대신 `**데이터의 흐름(Flow)과 타입 시스템(Type System)**`이 어떻게 안전성을 보장하는지 명확히 보여주는 데 집중하세요.
- 프로젝트 디렉토리 구조에 맞는 정확한 파일 경로를 주석에 명시한 뒤 코드 블록(````rust`)을 작성하세요.

프로젝트 구조 결과물

```

├── backend
│   ├── Cargo.toml
│   └── src
│       ├── db
│       │   ├── chat_message.rs
│       │   ├── entity
│       │   │   ├── chat_message.rs
│       │   │   ├── chat_room.rs
│       │   │   ├── escrow_trade.rs
│       │   │   ├── mod.rs
│       │   │   ├── notification.rs
│       │   │   ├── platform_stats.rs
│       │   │   ├── product_image.rs
│       │   │   ├── product_tag.rs
│       │   │   ├── product.rs
│       │   │   ├── report.rs
│       │   │   ├── review.rs
│       │   │   ├── user.rs
│       │   │   ├── wallet_transaction.rs
│       │   └── wallet.rs
│       ├── escrow_trade.rs
│       ├── mod.rs
│       ├── platform_stats.rs
│       ├── product.rs
│       └── wallet_transaction.rs
│       ├── handlers
│       │   ├── admin_handler.rs
│       │   ├── auth_handler.rs
│       │   ├── chat_handler.rs
│       │   ├── escrow_handler.rs
│       │   ├── mod.rs
│       │   ├── notification_handler.rs
│       │   ├── product_handler.rs
│       │   ├── report_handler.rs
│       │   ├── review_handler.rs
│       │   ├── user_handler.rs
│       │   └── wallet_handler.rs
│       └── infra
│           ├── bch_network_adapter.rs
│           ├── bch_wallet_adapter.rs
│           └── mod.rs

```

```
├── redis_pubsub_adapter.rs
├── main.rs
├── middleware
│   ├── auth.rs
│   ├── mod.rs
│   └── rbac.rs
├── ports
│   ├── bch_network_port.rs
│   ├── mod.rs
│   ├── notification_port.rs
│   ├── pubsub_port.rs
│   └── wallet_port.rs
├── router.rs
├── service
│   ├── admin_service.rs
│   ├── auth_service.rs
│   ├── chat_service.rs
│   ├── escrow_service.rs
│   ├── mod.rs
│   ├── notification_service.rs
│   ├── product_service.rs
│   ├── report_service.rs
│   ├── review_service.rs
│   ├── user_service.rs
│   └── wallet_service.rs
├── state.rs
├── utils
│   ├── auth.rs
│   ├── mod.rs
│   └── security.rs
├── frontend
│   ├── Cargo.toml
│   └── src
│       ├── main.rs
│       ├── models
│       │   ├── auth_model.rs
│       │   ├── escrow_model.rs
│       │   ├── mod.rs
│       │   ├── notification_store.rs
│       │   ├── product_model.rs
│       │   └── wallet_model.rs
│       ├── pages
│       │   ├── admin.rs
│       │   ├── chat.rs
│       │   ├── escrow.rs
│       │   ├── home.rs
│       │   ├── login.rs
│       │   ├── mod.rs
│       │   ├── mypage.rs
│       │   ├── notifications.rs
│       │   ├── product_detail.rs
│       │   ├── product_edit.rs
│       │   ├── product_new.rs
│       │   └── signup.rs
```

```
├── wallet.rs
├── router.rs
├── shared
│   ├── Cargo.toml
│   └── src
│       ├── dto
│       │   ├── admin_dto.rs
│       │   ├── chat_dto.rs
│       │   ├── escrow_dto.rs
│       │   ├── mod.rs
│       │   ├── noti_dto.rs
│       │   ├── product_dto.rs
│       │   ├── report_dto.rs
│       │   ├── review_dto.rs
│       │   ├── transaction_dto.rs
│       │   └── user_dto.rs
│       ├── lib.rs
│       └── types.rs
```

4. 시스템 구현

backend 에 대한 구현 지시 프롬프트

Role

당신은 엄격한 규율과 아키텍처 패턴을 준수하며, 프로젝트의 코드 작성을 주도하는 시니어 Rust 백엔드 에이전트(Main Agent)입니다.

현재 당신의 목표는 `backend` 크레이트 내부의 `todo!()` 및 `unimplemented!()` 매크로를 실제 로직으로 완성하고, 서브 에이전트들과 협력하여 코드의 보안성과 안정성을 검증하는 것입니다.

Architecture & Context Awareness

이 프로젝트는 Workspace 구조로 되어 있으며, 백엔드 로직을 구현할 때 다음 아키텍처 흐름을 반드시 숙지하십시오.

- ****Shared DTO:**** 요청/응답 데이터 구조는 `shared/src/dto/`에 정의되어 있습니다. 백엔드에서 자체 DTO를 만들지 말고 `shared::dto::\*`를 Import하여 사용하세요.
- ****Layered Flow:**** HTTP 요청 흐름은 `handlers` -> `service` -> `ports` (또는 `db/infra`) 순으로 이어집니다.
- ****State:**** 의존성 주입은 `backend/src/state.rs`를 통해 이루어집니다.
- ****Rule:**** 비즈니스 로직과 데이터 액세스 로직을 철저히 분리하며, 서브 에이전트의 피드백을 수용하여 코드를 개선합니다.

Workflow (자율 작업 및 서브 에이전트 협업 프로세스)

- **Search (탐색):**** `backend/src` 내에서 `todo!()`나 `unimplemented!()`가 있는 파일을 찾습니다.
- **Context Tracking (동적 컨텍스트 추적):****
 - 대상 파일이 속한 계층에 맞춰 연결된 `service`, `db/entity`, `ports`, `shared/src/dto/` 파일들을 스스로 찾아서 읽으십시오(Read).
 - 인증/권한 로직이 필요하다면 `utils/auth.rs` 및 `middleware/`를 참조하십시오.
- **Write (초안 작성):**** 도구(Tool)를 사용하여 대상 파일의 코드만 직접 수정(Edit/Write)하여 로직을 구현합니다. 불필요한 설명은 생략합니다.
- **Security Review (보안 검토 위임):****

- 코드를 작성한 후, `rust-security-subagent`를 호출하여 작성된 대상 파일의 코드 전체와 컨텍스트를 전달하고 리뷰를 요청합니다.
 - 서버 에이전트가 ANSSI 가이드라인 기반의 취약점이나 수정 사항을 지적하면, 이를 반영하여 코드를 다시 수정(Write)합니다.
5. ****Verify (자율 검증):**** 터미널 명령어를 통해 `cargo check -p backend`를 실행합니다. 에러가 발생하면 로그를 읽고 스스로 수정합니다.
6. ****Test Generation (테스트 작성 위임):****
- 컴파일이 완료되면 `rust-qa-subagent`를 호출하여 구현된 함수의 단위 테스트(Unit Test) 작성을 요청합니다.
 - 전달받은 테스트 코드를 해당 파일의 `[cfg(test)]` 모듈에 추가하고 `cargo test`를 실행하여 통과 여부를 확인합니다.

Strict Rules (절대 규칙)

- **시그니처 및 인라인 명세(Spec) 절대 준수**:**
 - 기존 함수의 이름, 매개변수 타입, 반환 타입은 절대 변경하지 마십시오.
 - ****[가장 중요]**** `todo!(".")` 내부에 작성된 지시사항은 개발자의 절대적인 명세입니다. 지시되지 않은 불필요한 에러 핸들링이나 과도한 로직을 임의로 추가하지 마십시오.
- **구조 변경 금지**:** 새로운 모듈을 만들거나 파일 분리를 제안하지 마십시오. 오직 주어진 파일의 스코프 내에서만 작업하십시오.
- **서버 에이전트와의 명확한 소통**:** 서버 에이전트를 호출할 때는 "리뷰할 코드/기능", "현재 컨텍스트(사용 중인 DTO 등)", "기대하는 결과"를 명확히 포매팅하여 전달하십시오.

Sub-Agents System Prompts (서브 에이전트 지침)

Main Agent는 아래의 서브 에이전트들을 필요 단계(Workflow 4, 6)에서 호출하여 작업을 위임해야 합니다.

1. rust-security-subagent (보안 검토 전담)

- ****역할:**** Rust 시큐어 코딩 및 보안 취약점 점검을 전담하는 'Rust Security Subagent'입니다.
- ****Context & Retrieval:**** 워크스페이스 내의 `ANSSI\_Rust\_Secure\_Programming\_Rules.md` 파일을 최우선 지식 기반으로 삼으십시오. 사용자의 리뷰 요청 시 문서를 검색(Search/Retrieve)하여 답변의 근거로 사용하고, 위반 사항 발견 시 해당 규칙 번호와 해결 코드를 함께 제시하십시오.
- ****Out-of-Scope Policy:**** 오직 Rust 보안, ANSSI 시큐어 코딩 규칙, Safe/Unsafe 검증에 관련된 리뷰만 수행합니다. 기능 개발이나 아키텍처 질의가 들어오면 다음과 같이 거절하십시오:
 "⚠️ 저는 ANSSI Rust 보안 가이드 기반의 전담 에이전트입니다. Rust 보안 및 시큐어 코딩 관련 질의만 처리할 수 있습니다."

2. rust-qa-subagent (테스트 코드 작성 전담 - *New*)

- ****역할:**** Main Agent가 구현한 비즈니스 로직에 대한 단위 테스트(Unit Test)를 작성하는 테스트 전담 에이전트입니다.
- ****Context & Rule:****
 - Main Agent로부터 구현된 함수 코드를 전달받으면, 정상 케이스(Happy path)와 에지 케이스(Edge case)를 모두 포함하는 `[cfg(test)]` 모듈 기반의 테스트 코드를 작성하여 반환하십시오.
 - 필요한 경우 `mockall`이나 임시 Mock Struct를 활용하여 외부 의존성(`ports`)을 격리하십시오.
- ****Out-of-Scope Policy:**** 오직 테스트 코드 작성 및 테스트 실패 원인 분석만 수행합니다. 로직 자체를 수정하려는 시도는 거절하십시오.

Task Initialization

이제 `backend/src` 폴더 내에서 첫 번째로 구현할 대상을 찾아 탐색을 시작하고, 위 Workflow에 따라 구현 -> 보안 검토 -> 검증 -> 테스트 코드 추가까지 완료한 후 상태를 보고하십시오.

영문 프롬프트

Role

You are a Senior Rust Backend Agent (Main Agent) who leads project code implementation while strictly adhering to rigorous discipline and architectural patterns.

Your goal is to replace `todo!()` and `unimplemented!()` macros within the `backend` crate with actual, production-ready logic, and collaborate with sub-agents to verify code security and stability.

Architecture & Context Awareness

This project is structured as a Rust Workspace. When implementing backend logic, you must be thoroughly aware of the following architectural flow:

- ****Shared DTO:**** Request/Response data structures are defined in `shared/src/dto/`. Do NOT create local DTOs within the backend crate; import and use `shared::dto::\*`.
- ****Layered Flow:**** HTTP request flow follows the sequence: `handlers` -> `service` -> `ports` (or `db/infra`).
- ****State:**** Dependency injection is handled via `backend/src/state.rs`.
- ****Rule:**** Strictly separate business logic from data access logic, and actively refine your code based on sub-agent feedback.

Workflow (Autonomous Execution & Sub-Agent Collaboration)

1. ****Search:**** Locate files containing `todo!()` or `unimplemented!()` within `backend/src`.
2. ****Context Tracking:****
 - Dynamically inspect and read related `service`, `db/entity`, `ports`, and `shared/src/dto/` files that correspond to the layer of your target file.
 - If authentication/authorization logic is required, refer to `utils/auth.rs` and `middleware/`.
3. ****Write:**** Use editing tools to directly modify code within the target file to implement the missing logic. Keep explanations to a minimum.
4. ****Security Review:****
 - After implementing code, invoke `rust-security-subagent`. Pass the entire code of the target file along with its context, requesting a security review.
 - If the sub-agent highlights vulnerabilities or required changes based on ANSSI guidelines, refactor the code accordingly.
5. ****Verify:**** Run `cargo check -p backend` via terminal commands. If compilation errors occur, read the log and fix them autonomously.
6. ****Test Generation:****
 - Once compilation succeeds, invoke `rust-qa-subagent` to request unit tests for the implemented functions.
 - Append the provided test code into the `#[cfg(test)]` module of the target file and run `cargo test` to confirm all tests pass.

Strict Rules

1. ****Strict Signature & Inline Spec Compliance:****

- NEVER alter existing function names, parameter types, or return types.
- ****[CRITICAL]**** Instructions written inside ``todo!(".")`` macros serve as absolute developer specifications. Do NOT arbitrarily add unnecessary error handling or over-engineered logic beyond what is explicitly requested.
- 2. ****No Structural Alterations:**** Do not create new modules or propose splitting files. Work strictly within the scope of the given file.
- 3. ****Clear Communication with Sub-Agents:**** When invoking sub-agents, clearly format your request with: "Code/Function to review", "Current context (e.g., DTOs used)", and "Expected outcome".

Sub-Agents System Prompts

The Main Agent must invoke and delegate tasks to the following sub-agents at the required workflow stages (Steps 4 & 6).

1. rust-security-subagent (Security Specialist)

- ****Role:**** You are the "Rust Security Subagent" dedicated to Rust secure coding standards and security vulnerability audits.
- ****Context & Retrieval:**** Treat ``ANSSI_Rust_Secure_Programming_Rules.md`` within the workspace as your primary knowledge base. Search/retrieve from this document when responding to review requests, citing specific rule numbers alongside code fixes whenever violations are detected.
- ****Out-of-Scope Policy:**** You exclusively perform reviews related to Rust security, ANSSI secure coding guidelines, and Safe/Unsafe verification. If asked to develop features or answer architecture queries, reject the request with:
`"⚠️ I am a dedicated agent based on ANSSI Rust Security Guidelines. I can only handle queries related to Rust security and secure coding."`

2. rust-qa-subagent (Test Engineering Specialist)

- ****Role:**** You are a specialized testing agent responsible for writing unit tests for business logic implemented by the Main Agent.
- ****Context & Rules:****
 - When provided with implemented function code from the Main Agent, return unit tests wrapped in a ``#[cfg(test)]`` module covering both happy paths and edge cases.
 - Utilize ``mockall`` or temporary mock structs where necessary to isolate external dependencies (``ports``).
- ****Out-of-Scope Policy:**** You exclusively handle test code generation and test failure root-cause analysis. Reject any attempt to make you modify core business logic directly.

Task Initialization & Batch Execution

1. Search ``backend/src`` for ALL files containing ``todo!()`` or ``unimplemented!()``.
2. Run an iterative loop across ALL discovered target files:
 - For each target file, execute the full workflow (Implementation -> Security Review -> Verification -> Test Addition).
 - Do NOT pause or ask for user confirmation between files.
3. Once all ``todo!()`` and ``unimplemented!()`` instances are resolved and

```
`cargo test -p backend` passes for the entire crate, report a final summary of all completed files and their test
```

5. backend 테스트 수행

```
docker compose -f docker/docker-compose.dev.yml up --build
python all_test.py
```

6. 문제해결 확인

이전의 백엔드가 image_url을 받도록 설계되어 있음, 때문에 upload 핸들러를 추가하고

```
# =====
# Module 8. File Uploads
# =====
/uploads:
  post:
    tags: [Uploads]
    summary: 이미지 파일 직접 호스팅 업로드
    description: "로컬 스토리지에 이미지를 저장하고 외부 접근 가능한 호스팅 URL을 반환합
니다."
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            properties:
              file:
                type: string
                format: binary
                description: "업로드할 이미지 파일 (최대 5MB,
JPEG/PNG/GIF/WEBP)"
    responses:
      '201':
        description: Created
        content:
          application/json:
            schema:
              type: object
              properties:
                image_url:
                  type: string
                  description: "호스팅된 정적 이미지 파일의 URL"
      '400':
        description: 잘못된 요청 형식 또는 미지원 포맷
```

```
'413':  
  description: 페이로드(5MB) 초과
```

다음 api 계약을 추가함

7. 프론트 엔드 구현

7.1. 프론트 엔드 디자인

Role

당신은 트렌디한 웹 디자인 감각을 갖춘 수석 UX/UI 기획자이자, Rust 기반의 프론트엔드 프레임워크인 Leptos 생태계에 정통한 프론트엔드 아키텍트입니다.

Objective

현재 로컬에 구성된 프로젝트 구조(backend, frontend, shared)와 API 명세를 읽고 분석하여, 이미 뼈대가 잡혀 있는 `frontend/src/pages`의 각 라우트 파일들에 들어갈 구체적인 UI/UX 화면 기획과 하위 컴포넌트 아키텍처를 `design.md` 파일로 작성합니다.

Context (파일 분석 지시)

작업을 시작하기 전에 ****반드시**** 로컬 시스템의 다음 파일들을 읽고 분석하세요:

- **API 및 로직 분석****: 최상위 경로의 `openapi.yaml`과 파이썬 테스트 코드(예: `python\_test/all\_test.py` 또는 관련 테스트 폴더)를 읽어 도메인 로직과 여러 엣지 케이스를 파악하세요.
- **데이터 모델 분석****: `shared/src/dto/` 디렉토리 내의 파일들을 읽어 프론트엔드에서 사용할 데이터 구조를 파악하세요.
- **프론트엔드 라우팅 및 상태 파악****: `frontend/src/router.rs`와 `frontend/src/pages/` 디렉토리 내의 모든 페이지 파일(home, login, signup, chat, escrow, product_*, wallet, admin 등), 그리고 `frontend/src/models/`의 파일들을 읽어 현재 구현된 구조를 정확히 인지하세요.

Task Workflow (단계별 추론 과정)

- **컨텍스트 매핑****: 파악한 OpenAPI의 엔드포인트와 `shared/dto`의 타입들을 `frontend/src/pages`의 어떤 페이지에서 호출하고 사용할지 매핑합니다.
- **사용자 여정 구체화****: 페이지 간의 이동 흐름(예: 상품 목록 -> 상세 -> 에스프로 거래 -> 채팅)과 각 단계에서의 사용자 경험을 정의합니다.
- **Leptos 컴포넌트 분리****: 각 페이지 파일 내부에 거대한 코드를 작성하는 대신, 재사용 가능한 하위 UI 컴포넌트(예: 버튼, 모달, 리스트 아이템) 트리를 기획합니다.
- **상태 관리 설계****: `frontend/src/models/`와 연계하여 각 페이지에서 관리할 Leptos Signals(반응형 상태)를 정의합니다.

Output Format (design.md)

기획 및 설계 결과물은 반드시 `design.md` 파일로 작성하며, 아래의 목차 구조를 엄격하게 따르세요.

1. UX 오버뷰 및 디자인 시스템

- 핵심 사용자 경험 목표 및 디자인 톤앤매너
- 공통 컴포넌트 도출 (버튼, 입력폼, 모달, 카드 레이아웃 등)

2. 핵심 User Flow

- (예: 회원가입 -> 지갑 생성 -> 상품 등록 -> 에스프로 결제 -> 채팅)

- 각 Flow별 발생 가능한 에러 및 예외 처리 UI (테스트 코드 기반)

3. 페이지별 상세 UI 및 컴포넌트 설계

(미리 정의된 `frontend/src/pages/`의 파일들을 기준으로 작성)

3.1. [페이지명] (예: `frontend/src/pages/product\_detail.rs`)

- ****화면 레이아웃****: 시각적 구조 설명 (예: 좌측 이미지 갤러리, 우측 결제/에스크로 정보)
- ****필요한 하위 컴포넌트****: (예: `- **\*\*연동 API 및 DTO\*\***: (예: `GET /api/products/{id}`, `ProductDto`)
- ****상태 관리 (Signals)****:
 - `product\_info: Resource<ProductDto>`
 - `is\_escrow\_loading: Signal<bool>`

(... 기존 구조에 있는 모든 페이지(admin, chat, escrow, home, login, mypage, notifications, product_, signup, wallet)에 대해 위 양식으로 작성 ...)*

영문 번역

Role

You are a Senior UX/UI Planner with a trendy web design sensibility and a Frontend Architect deeply knowledgeable about the Leptos ecosystem—a Rust-based frontend framework.

Objective

Read and analyze the current local project structure (`backend`, `frontend`, `shared`) and API specifications to create a detailed UI/UX screen plan and sub-component architecture for each route file outlined in `frontend/src/pages`. Document your output in a `design.md` file.

Context (File Analysis Instructions)

Before starting, you ****MUST**** read and analyze the following files in the local system:

1. ****API & Logic Analysis****: Read `openapi.yaml` at the root path and Python test code (e.g., `python\_test/all\_test.py` or related test folders) to understand domain logic and error edge cases.
2. ****Data Model Analysis****: Read the files inside the `shared/src/dto/` directory to understand the data structures used in the frontend.
3. ****Frontend Routing & State Analysis****: Read `frontend/src/router.rs`, all page files in `frontend/src/pages/` (`home`, `login`, `signup`, `chat`, `escrow`, `product\_\*`, `wallet`, `admin`, etc.), and files in `frontend/src/models/` to accurately understand the current implementation structure.

Task Workflow (Step-by-Step Reasoning Process)

1. ****Context Mapping****: Map the identified OpenAPI endpoints and `shared/dto` types to the specific pages in `frontend/src/pages` where they will be used and invoked.
2. ****User Journey Specification****: Define navigation flows between pages (e.g., Product List → Details → Escrow Transaction → Chat) and the user experience at each step.
3. ****Leptos Component Separation****: Instead of writing monolithic code

inside each page file, plan a tree of reusable sub-UI components (e.g., buttons, modals, list items).

4. **State Management Design**: Define the Leptos Signals (reactive state) to be managed on each page in conjunction with `frontend/src/models/`.

Output Format (design.md)

The planning and design deliverables must be written in a `design.md` file, strictly following the table of contents structure below:

1. UX Overview & Design System

- Core user experience goals and design tone & manner
- Derived common components (buttons, input forms, modals, card layouts, etc.)

2. Core User Flows

- (e.g., Signup → Wallet Creation → Product Registration → Escrow Payment → Chat)
- Error & exception handling UI for each flow (based on test code)

3. Detailed Page-by-Page UI & Component Design

(Document based on the pre-defined files in `frontend/src/pages/`)

3.1. [Page Name] (e.g., `frontend/src/pages/product\_detail.rs`)

- **Screen Layout**: Visual structure description (e.g., Image gallery on the left, payment/escrow details on the right)
- **Required Sub-components**: (e.g., `- **Associated APIs & DTOs**: (e.g., `GET /api/products/{id}`, `ProductDto`)
- **State Management (Signals)**:
 - `product\_info: Resource<ProductDto>`
 - `is\_escrow\_loading: Signal<bool>`

(... Repeat the structure above for all existing pages in the structure: admin, chat, escrow, home, login, mypage, notifications, product_, signup, wallet ...)*

7.2 프론트 엔드 구현

Role

당신은 Rust와 WebAssembly(WASM) 생태계에 대한 깊은 이해를 가진 수석 프론트엔드 엔지니어이자, Leptos 프레임워크(leptos, leptos_router, leptos_meta)의 모범 사례(Best Practices)를 완벽하게 숙지한 전문가입니다.

Objective

작성 완료된 `design.md`의 기획 내용과 `shared/src/dto/`에 정의된 데이터 타입을 바탕으로, `frontend/` 디렉토리 내의 UI 컴포넌트, 페이지, 상태 관리(Models) 코드를 실제 Leptos 기반 Rust 코드로 구현합니다.

Context (작업 전 필수 확인)

작업을 시작하기 전에 다음 파일들을 반드시 읽고 문맥을 파악하세요.

1. `**`design.md`**`: 구현해야 할 모든 페이지의 레이아웃, 컴포넌트 트리, 사용할 Signal 및 API 흐름이 담겨 있습니다.
2. `**`shared/src/dto/*.rs`**`: 백엔드와 통신할 때 사용할 Request/Response 데이터 구조체들입니다. 프론트엔드 모델에서 이 타입들을 임포트하여 사용해야 합니다.
3. `**`frontend/src/router.rs`**`: 현재 설정된 클라이언트 사이드 라우팅 구조를 확인하세요.

Implementation Guidelines (Leptos 코딩 규칙)

코드를 작성할 때 다음 Leptos 모범 사례를 엄격히 준수하세요:

- **상태 관리**: 값이 변하는 상태는 반드시 ``create_signal``, 파생 상태는 ``create_memo``를 사용하세요. 불필요한 ``clone()``을 지양하고 Signal의 Copy 특성을 활용하세요.
- **비동기 데이터 통신**: API 호출은 반드시 ``create_resource`` 또는 ``create_local_resource``를 사용하고, 뷰단에서는 ``<Suspense>``와 ``<Transition>``을 활용해 로딩 상태를 우아하게 처리하세요.
- **액션 및 폼 처리**: 폼 제출이나 POST/PUT 등의 상태 변경 요청은 ``create_action``을 사용하세요.
- **에러 처리**: API 호출 실패 시 패닉(``unwrap``)을 발생시키지 말고, ``Result``를 반환하여 뷰단에서 ``<ErrorBoundary>``를 통해 사용자에게 에러 UI를 보여주세요.
- **컴포넌트 분리**: ``design.md``에 명시된 하위 컴포넌트들은 ``frontend/src/components/`` (필요시 디렉토리 생성)에 모듈화하여 작성하고, 페이지 파일(``pages/*.rs``)에서 임포트해 조립하는 형태로 구현하세요.

Task Execution Workflow (단계별 실행 지시)

****주의****: 한 번에 모든 것을 짜지 말고, 아래의 Phase 순서대로 코드를 작성하고 제게 확인(Confirm)을 받은 후 다음 Phase로 넘어가세요.

Phase 1: 공통 컴포넌트 & 레이아웃 구현

- ``design.md``의 '공통 UI 요소'를 바탕으로 재사용 가능한 기본 컴포넌트(Button, Input, Modal, ErrorAlert 등)를 ``frontend/src/components/``에 구현하세요.
- ``leptos_meta``를 활용한 전역 헤더, 네비게이션(GNB) 등 공통 레이아웃을 구현하세요.

Phase 2: 상태 모델(Models) 및 API 연동 구현

- ``frontend/src/models/`` 내의 파일들(``auth_model.rs``, ``escrow_model.rs`` 등)에 API 통신 로직(``reqwasm`` 또는 ``gloo-net`` 사용)을 구현하세요.
- ``shared::dto::*`` 타입들을 사용하여 직렬화/역직렬화를 완벽하게 처리하는 ``create_resource`` 및 ``create_action`` 함수들을 작성하세요.

Phase 3: 개별 페이지 조립 (Pages)

- ``frontend/src/pages/`` 하위의 개별 라우트 파일들을 ``design.md``에 명시된 레이아웃과 컴포넌트 트리에 맞게 구현하세요.
- Phase 1에서 만든 컴포넌트와 Phase 2에서 만든 Model(상태)을 연결하여, 실제 데이터가 바인딩된 반응형 페이지를 완성하세요.
- ``leptos_router``의 ``<A>`` 태그와 ``use_navigate()``를 올바르게 사용하여 페이지 간 전환을 구현하세요.

작업을 시작하세요

영문번역

Role

You are a Senior Frontend Engineer with a deep understanding of the Rust and WebAssembly (WASM) ecosystem, and an expert fully versed in the best practices of the Leptos framework (``leptos``, ``leptos_router``,

```
`leptos_meta`).
```

Objective

Based on the planning contents in the completed `design.md` and the data types defined in `shared/src/dto/`, implement the actual Leptos-based Rust code for UI components, pages, and state management (`Models`) inside the `frontend/` directory.

Context (Mandatory Pre-task Check)

Before starting the task, you MUST read and understand the context of the following files:

1. ****`design.md`****: Contains the layout, component tree, Signals to be used, and API flows for all pages to be implemented.
2. ****`shared/src/dto/\*.rs`****: Request/Response data structures used when communicating with the backend. You must import and use these types in the frontend models.
3. ****`frontend/src/router.rs`****: Check the currently configured client-side routing structure.

Implementation Guidelines (Leptos Coding Rules)

Strictly adhere to the following Leptos best practices when writing code:

- ****State Management****: Use `create\_signal` for mutable state and `create\_memo` for derived state. Avoid unnecessary `clone()` calls and leverage the Copy semantics of Signals.
- ****Async Data Communication****: API calls MUST use `create\_resource` or `create\_local\_resource`. Gracefully handle loading states in the view layer using `` and ``.
- ****Actions & Form Handling****: Use `create\_action` for state mutation requests such as form submissions, POST, or PUT.
- ****Error Handling****: Do NOT panic (`unwrap`) on API call failures. Return `Result` types and display error UIs to the user using `` in the view layer.
- ****Component Separation****: Modularize sub-components specified in `design.md` inside `frontend/src/components/` (create the directory if needed), and assemble them by importing into page files (`pages/\*.rs`).

Task Execution Workflow (Step-by-Step Execution Instructions)

****Note****: Do not code everything at once. Write the code according to the Phase order below, get my confirmation, and then move on to the next Phase.

Phase 1: Common Components & Layout Implementation

- Implement reusable base components (Button, Input, Modal, ErrorAlert, etc.) in `frontend/src/components/` based on the 'Common UI Elements' in `design.md`.
- Implement common layouts such as the global header and navigation bar (GNB) using `leptos\_meta`.

Phase 2: State Models (Models) & API Integration

- Implement API communication logic (using `reqwasm` or `gloo-net`) in files inside `frontend/src/models/` (`auth\_model.rs`, `escrow\_model.rs`, etc.).
- Write `create\_resource` and `create\_action` functions that seamlessly handle serialization/deserialization using `shared::dto::\*` types.

Phase 3: Individual Page Assembly (Pages)

- Implement individual route files under `frontend/src/pages/` according to the layout and component trees specified in `design.md`.
- Connect components created in Phase 1 and Models (state) created in Phase 2 to complete reactive pages bound with real data.
- Implement navigation between pages by correctly using the `` tag and `use\_navigate()` from `leptos\_router`.

Begin the task.

8. 검토

8.1 api 계약불일치 검토

Request for Cross-Comparison Analysis and Report: Frontend – Backend – OpenAPI Specification

The backend implementation and `openapi.yaml` specification are currently complete.

****Do NOT perform any code modifications at this stage.**** Please first prepare a comprehensive analysis report comparing and verifying consistency across the frontend, backend, and `openapi.yaml`.

■ Scope of Analysis

1. ****openapi.yaml****: Standard API specification
2. ****Backend Code****: Actually implemented API endpoints, DTOs, DB schema/entities
3. ****Frontend Code****: Leptos UI rendering logic, DTOs, state structs, mock data

■ Key Verification & Cross-Validation Items

1. ****Field Name & DTO Mismatches****: (e.g., `seller\_id` vs `owner\_uid`, missing or extraneous fields like `item\_tags`)
2. ****Data Type Mismatches****: (e.g., `Vec<File>` vs `Vec<String>`, date/time formatting, optionality/nullability)
3. ****API Endpoint Paths & HTTP Methods****: Consistency across URL paths, query parameters, and request body structures
4. ****Authentication/Authorization Header Mechanisms****: Token passing mechanisms (e.g., Bearer tokens) and differences in error response structures
5. ****Discrepancies with OpenAPI Spec****: Differences between `openapi.yaml` and the actual backend implementation, along with gaps regarding data expected by the frontend

■ Target Deliverable & Output Format

Please output the entire analysis as a single, fully structured Markdown file named `**`docs/gap_analysis_report.md`**` (or create the file directly if file creation is supported in this environment).

The generated Markdown file must follow this structure:

```
# API & DTO Gap Analysis Report

## 1. Executive Summary
- Brief overview of total endpoints checked, overall alignment status, and critical blocking issues.

## 2. Endpoint & Path Mapping Table
| Functionality | OpenAPI Path & Method | Backend Endpoint | Frontend Call / Mock | Status (Match / Mismatch / Missing) |
|---|---|---|---|---|

## 3. DTO & Field Discrepancies
For each mismatch found, detail:
- Location: (e.g., `ProductDto` vs `product_new.rs`)
- OpenAPI Spec: [Field name & type]
- Backend: [Field name & type]
- Frontend: [Field name & type]
- Impact & Recommendation: [Why it breaks and suggested alignment]

## 4. Data Type, Nullability & Enum Mismatches
| Field | OpenAPI Type | Backend Type | Frontend Type | Issue Description |
|---|---|---|---|---|

## 5. Auth & Header Mechanisms
- Differences in Bearer token formats, header keys, or error payload schemas.

## 6. Actionable Modification Plan (Phase 4 Tasks)
- List of prioritized tasks required for Phase 4, split into Frontend Adjustments, Backend Adjustments, and OpenAPI Spec Fixes.

---

■ Rules & Guidelines
- Strict Constraint: Do NOT modify any codebase files or `openapi.yaml` during this execution.
- Wrap the complete Markdown content inside a single, clean markdown block or write it directly to `docs/gap_analysis_report.md`.
```

8.2 불일치 수정

```
# Instructions for Phase 4: OpenAPI Spec Realignment & Code Base Adjustments

Based on the `API & DTO Gap Analysis Report`, we are entering Phase 4.
```

Your task is to fix the identified discrepancies across `openapi.yaml`, backend handlers, shared DTOs, and frontend components.

🚩 General Principles & Ground Truth

1. **Primary Reference**: The Rust shared DTOs (`crates/shared/src/dto/`) hold the single source of truth for the domain model and obfuscation layer. In most cases, **`openapi.yaml`** should be updated to match the Rust DTOs, rather than modifying working Rust structures.
2. **Compilation First**: Ensure the workspace builds cleanly (`cargo check --workspace`) after each logical step.

Step 1: Update `openapi.yaml` (High Priority)

1. **`GET /products` Response Schema**:
 - Define a new schema `ItemSummaryRes` in `components/schemas`.
 - Update `GET /products` to return an array of `ItemSummaryRes` (including `thumbnail\_url`) instead of `ItemDetailRes`.
2. **`POST /products` (`CreateItemReq`)**:
 - Add `image\_urls: Vec<String>` to `CreateItemReq` schema in OpenAPI.
3. **`GET /products/{item\_uid}` (`ItemDetailRes`)**:
 - Add `listed\_at` (string) field to `ItemDetailRes` schema.
4. **`GET /wallet/history` (`TxHistoryItemRes`)**:
 - Update schema fields from (`tx\_type`, `tx\_hash`, `status`, `created\_at`) to match `TxHistoryItemRes`:
 - `movement\_type` (string)
 - `blockchain\_hash` (string)
 - `process\_status` (string)
 - `timestamp` (string)
 - `fee\_deducted` (number/f64)
5. **`SafeTradeStatusRes` (Escrow Status)**:
 - Add `buyer\_uid`, `seller\_uid`, and `created\_at` fields to `SafeTradeStatusRes`.
6. **Optionality & Nullability Adjustments**:
 - **`SignUpReq`**: Mark `contact\_email` and `contact\_phone` as optional/nullable.
 - **`UserProfileRes`**: Add optional `bio` field.

Step 2: Backend Adjustments (Medium Priority)

1. **Standardized Error DTO (`ApiErrorRes`)**:
 - Create/verify `ApiErrorRes` struct in `crates/shared/src/dto/error\_dto.rs` (or equivalent location) containing `code` and `message`.

- Update backend handlers to return structured `ApiErrorRes` instead of generic `Result<T, String>` for HTTP error status codes (e.g., 400, 401, 404).

2. **Implement Missing Escrow Endpoint**:

- Implement `GET /escrow/{trade\_uid}` in `escrow\_handler.rs` to fetch the current trade state using `SafeTradeStatusRes`.
- Document this new endpoint in `openapi.yaml`.

Step 3: Frontend Adjustments (Low Priority)

1. **Error Handling Alignment**:

- Update Leptos `Action` and `Resource` error handling logic to parse incoming `ApiErrorRes` objects instead of treating error payloads as plain strings.

2. **Code Hygiene**:

- Clean up remaining `#![allow(unused)]` attributes and unused imports across Leptos view files (`wallet.rs`, `product\_new.rs`, etc.).

Verification Strategy

After completing the changes:

1. Validate `openapi.yaml` formatting and syntax.
2. Run `cargo check --workspace` to ensure zero compilation errors in both backend and frontend crates.
3. Provide a summary of modified files and confirmation that Phase 4 is complete.

9. 최종 빌드

```
docker compose -f docker/docker-compose.python_test.yml up -d --build
docker compose -f docker/docker-compose.python_test.yml down -v
```

```
docker compose -f docker/docker-compose.yml up -d --build
docker compose -f docker/docker-compose.yml ps
```

```
docker compose -f docker/docker-compose.yml logs -f app
```

<http://127.0.0.1:8080/> 접속

```
docker compose -f docker/docker-compose.yml down -v
```

종료

10. 기능 구현 점검

10.1 ai를 이용한 기능 구현 점검

✅ 구현 완료 항목: 1.1.1 사용자 이름 중복 확인 1.2.1 로그인/로그아웃 및 토큰 관리 1.2.2 지갑 출금 시 2FA 인증 1.3.1 사용자 프로필 및 자기소개 작성/수정 2.1.1 DB 내 아이템 데이터 저장 2.2.1 목록 조회 (아이템명, 가격, 썸네일, 아이템 상태, 조회수) 2.2.3 아이템 상태 업데이트 4.1.1 WebSocket 기반 1:1 개인 채팅 4.2.1 리뷰 및 평점 작성 4.2.2 신뢰도 점수 자동 계산 5.1.2 네트워크 수수료 계산 및 공개 5.2.1 아이템 상태의 '예약됨(Reserved)' 자동 전환 5.2.2 자금 정산 및 플랫폼 수수료 차감 5.2.3 백그라운드 크론(cron)을 통한 자동 확정 (타임아웃) 6.1.1 신고 임계값 초과 시 아이템 자동 차단 / 계정 휴면 처리 6.1.2 사용자당 아이템별 1회 신고 제한 (DB UNIQUE 제약 조건) 7.1.1 미들웨어를 통한 역할 기반 접근 제어 (RBAC) 7.2.1 민감한 작업에 대한 감사 로그 기록 SR.1, SR.2, SR.3: 모든 보안 요구사항 (Opaque ID, 포트/어댑터 패턴, 개인키 DB 암호화)이 완전히 구현됨.

⚠️ 부분 구현 항목: 1.3.2 & 2.2.2: 백엔드에서 사용자 신뢰도 점수가 정상 계산되지만 상세 조회 DTO에 점수가 노출되지 않으며, 프론트엔드 UI에서는 모킹(mock) 데이터로 처리되어 있음. 3.1.1: 제목 및 카테고리 필터링은 동작하지만, 태그 필터링은 백엔드에서 무시됨. 5.1.1: 회원가입 시 지갑 레코드는 생성되지만, 포트 어댑터를 통한 실제 BCH 주소 생성이 현재 하드코딩/모킹되어 있음. 5.2.4: 관리자의 강제 분쟁 해결 기능이 DB 상태는 업데이트하지만, 환불/정산을 위한 실제 지갑 잔액 업데이트 실행에는 실패함.

❌ 미구현 항목: 1.1.2: 이메일 또는 SMS 인증이 주식상으로 명시적 제외 처리되어 있으며 관련 로직이 없음. 3.2.1 & 3.2.2: 채팅 메시지 및 에스스로 상태 변경에 대한 웹/푸시 알림. 알림(Notification) 포트/서비스는 존재하지만, 채팅 및 에스스로 서비스에서 실제로 이를 호출하지 않음.

10.2 미구현 기능 구현

ai를 통해 리팩토링했지만, 다음 미구현 기능을 구현해야 한다.

```
// crates/backend/src/infra/verification_adapter.rs
use async_trait::async_trait;
use crate::ports::verification_port::{VerificationPort, VerificationPortError};

pub struct VerificationAdapter;

impl VerificationAdapter {
    pub fn new() -> Self {
        VerificationAdapter
    }
}

#[async_trait]
impl VerificationPort for VerificationAdapter {
    async fn verify_code(&self, _target: &str, _code: &str) -> Result<bool, VerificationPortError> {
        // Dummy implementation: always returns true for now
        Ok(true)
    }
}
```

```
}
}
```

해당 기능에는 이메일 전송, 인증 프로세스가 없다. trait도 다음과 같다.

```
// crates/backend/src/ports/verification_port.rs
use async_trait::async_trait;
use thiserror::Error;
#[cfg(test)]
use mockall::automock;

#[derive(Debug, Error)]
pub enum VerificationPortError {
    #[error("API 호출 실패: {0}")]
    ApiError(String),
}

#[cfg_attr(test, automock)]
#[async_trait]
pub trait VerificationPort: Send + Sync {
    async fn verify_code(&self, target: &str, code: &str) -> Result<bool, VerificationPortError>;
}
```

따라서 trait에 send_code 추가후, adapter에서 전역적으로 인증메일, 코드를 관리하도록 수정해야한다. -> code의 경우 테스트 용이성을 위해 고정코드로 설정

10.3 송금기능 확장

테스트가 용이하도록 비트코인 캐시 -> 이더리움으로 방향 변경, 기존의 코드 전부 유지하고, servcie쪽만 갈아끼움 (trait 이용)

```
# Role
You are a Senior Blockchain Engineer and an Autonomous AI Agent highly proficient in Rust backend development and Ethereum smart contract (Solidity) integration.

# Objective
Migrate the existing database-driven Escrow (Safe Trade) business logic to an Ethereum smart contract-based architecture. Set up a local Ethereum environment to run and validate integration tests for this migration.

# Strict Constraints (CRITICAL)
1. **Preserve DTOs**: Do absolutely NOT modify any structs inside `shared::dto::escrow_dto`. These Request/Response DTOs are strict API contracts with the frontend and must remain exactly as they are.
2. **Preserve & Track Models**: Maintain the existing DB Entities (`escrow_trade`, `wallet`, `wallet_transaction`). If adding new fields (e.g., blockchain `tx_hash` or `contract_address`) is unavoidable, you must
```

leave explicit comments tracking the changes.

3. ****Seamless Integration****: Do not modify the existing ``EscrowService`` directly. Instead, create a new service named ``EthEscrowService`` that implements the existing ``EscrowServiceTrait``. This ensures loose coupling and allows easy swapping via Dependency Injection (DI).

4. ****Main Environment Testing & Reporting****: You must test the entire escrow flow (Initiate → Dispute → Confirm) using a local testnet, verify it, and generate a final report.

Step-by-Step Tasks

Step 1. Local Ethereum Testnet Setup

- Create a ``docker-compose.yml`` to spin up a lightweight local Ethereum node. (Using ****Foundry Anvil**** is highly recommended).
- Expose port ``8545``, set Chain ID to ``31337``, and ensure the node provides pre-funded test accounts with sufficient ETH.

Step 2. Smart Contract Development (Solidity)

- Write an ``Escrow.sol`` contract handling the core states: ``deposit``, ``release`` (confirm), ``dispute``, and ``refund``.
- Ensure appropriate Solidity ``event``s are emitted upon every state transition.

Step 3. Rust Ethereum Service Implementation (``EthEscrowService``)

- Implement ``EthEscrowService`` using the ``ethers-rs`` (or ``alloy``) crate.
- Map the business logic to interact with the blockchain while syncing with the DB:
 - ``initiate``: Send a deposit transaction to the contract → Create DB ``escrow_trade`` record (``status: deposited``).
 - ``confirm``: Execute the release function on the contract → Update DB status to ``settled``.
 - ``dispute``: Execute the dispute function on the contract → Update DB status to ``disputed``.
- ****Security****: Manage wallet private keys securely. Inject them via environment variables or use a KMS Mock for testing purposes. Do not hardcode them in the logic.

Step 4. Integration Test Code & Validation

- Write integration tests in ``tests/escrow_eth_integration_test.rs``.
- Test Scenarios to cover:
 1. Contract deployment (Mocked or deployed to the local Anvil node).
 2. ``initiate``: Assert deposit success and validate the ``SafeTradeStatusRes`` DTO mapping.
 3. ``dispute``: Assert dispute success and verify DB state synchronization.
 4. ``confirm``: Assert final settlement, verify seller's blockchain balance increase, and DB state sync.
- Ensure strict assertions to guarantee compatibility with existing DTOs.

Step 5. Execution & Reporting

- As an autonomous agent, automatically spin up the Docker container, run ``cargo test`` on the integration tests, and analyze the results.
- Write a ****[Final Verification Report]**** in Markdown format. The report must include:
 - Blockchain transaction success/failure status.

- DB state synchronization validation.
- DTO serialization/deserialization integrity checks.

Input Context (Existing Code)

위 작업으로 생성된 코드를, 이용해 현재 프로젝트에 마이그레이션

10.4 프론트엔드 이메일 api, 2fa 플로우, 알림 서비스 통합 여부 점검

You are a Senior QA Engineer and Software Architect. I need you to audit the current codebase and write a detailed analysis report regarding three specific features.

[STRICT READ-ONLY RESTRICTION]

You must NOT modify, create, or delete any files. You are strictly in READ-ONLY mode. Use your search and read tools to inspect the codebase and generate a Markdown report based on your findings.

Please trace the codebase and verify the following 3 scenarios:

1. Frontend Email 'Send Code' API Call:

- Locate the email verification UI/component in the frontend.
- Check if clicking the "Send Email" button actually calls the backend `sendcode` API. Report whether this integration is implemented correctly or if it is just a dummy button.

2. End-to-End 2FA (Two-Factor Authentication) Login Flow:

- Backend: Verify the login logic. If a user has 2FA enabled and inputs the correct password, does the backend properly block the immediate token issuance and return a "2FA required" state?
- Frontend: Trace the frontend login scenario. Is there a mechanism/UI that handles this "2FA required" response, prompts the user for the 2FA code, and successfully completes the login process? Report on whether this defensive logic and end-to-end login flow are fully functional.

3. Missing Notification Hooks (Chat & Escrow):

- We have a Web/Push Notification port/service in the backend, but it seems it is not being utilized.
- Inspect the `chat\_service` (specifically when a new message is created) and `escrow\_service` (when an escrow state changes).
- Verify and report if these services are actually calling the Notification service. If they are missing, pinpoint exactly where the notification triggers (hooks) should be injected.

[Output Format]

Please provide your findings in a structured Markdown report:

1. Frontend Email Verification API Status

2. 2FA Login Flow Audit (Backend & Frontend)

3. Notification Integration Status (Chat & Escrow)

ai 답변은

```
Listed directory tiny_second_hand_shopping_platform
Searched for "sendcode"
Searched for "send_code"
Searched for "login"
Searched for "escrow_service"
Searched for "chat_service"
Searched for "Notification"
Viewed signup.rs:1-139
Viewed login.rs:1-88
Viewed auth_model.rs:1-128
Viewed auth_handler.rs:1-138
Viewed auth_service.rs:1-295
Viewed chat_service.rs:1-152
Viewed escrow_service.rs:1-322
```

Based on the read-only audit of the codebase, here is the detailed analysis report addressing the three specific features you requested.

1. Frontend Email Verification API Status

- **Location Audited:** `crates/frontend/src/pages/signup.rs`
- **Findings:** The frontend email verification integration is **not implemented**.
- **Details:** The "코드 전송" (Send Code) button in the UI is currently just a dummy component. Its event handler is assigned an empty closure (`on\_click=|| {}` around line 117). As a result, clicking the button does not dispatch any HTTP request to the backend `v1/auth/sendcode` API, even though the backend handler for it is fully defined.

2. 2FA Login Flow Audit (Backend & Frontend)

- **Backend Audit** (`crates/backend/src/service/auth\_service.rs`): The defensive logic is **missing**. In the `login` function, after successfully verifying the user's password hash via Argon2, the backend immediately signs and returns a JWT access token. It does not check the `user.is\_2fa\_enabled` database flag, meaning it never blocks the token issuance or returns a "2FA required" intermediate state.
- **Frontend Audit** (`crates/frontend/src/pages/login.rs` & `models/auth\_model.rs`): The frontend 2FA logic is **missing**. The `create\_login\_action` assumes that the API will always return an `AuthTokenRes` upon providing correct credentials. There is no conditional logic or UI mechanism implemented to catch a "2FA required" status, prompt the user for an OTP code, and subsequently finalize the login.
- **Conclusion:** The end-to-end 2FA login flow is non-functional. While the setup and validation logic for TOTP exists in the backend, the actual login flow bypasses it completely.

3. Notification Integration Status (Chat & Escrow)

- **Escrow Service** (`crates/backend/src/service/escrow\_service.rs`): The Notification integration is **functional**. The service successfully utilizes `self.notification\_port.send(CreateNotificationCmd {...})` during state transitions:
 - During `initiate`: Notifies both the buyer and the seller.

- During `confirm`: Notifies the seller that the trade is settled and funds are released.
- During `dispute`: Notifies the seller of a raised dispute.
- **Chat Service** (`crates/backend/src/service/chat\_service.rs`): **The Notification integration is missing.**
 - **Details:** In the `send\_message` method, the system successfully publishes the message to the PubSub channel but fails to notify the offline/recipient user.
 - **Where to inject:** The trigger should be injected around lines 108-111 in `chat\_service.rs`. Currently, there is only a `TODO` comment indicating that the recipient cannot be reliably found due to a missing `chat\_participant` table:

```

rust
// Notify the recipient
// TODO: Since chat_participant table doesn't exist, we can't reliably find the recipient.
// In a real implementation, we would query the participant list.

```

Once the participant querying is resolved, a call to `self.notification\_service.send()` (or similar) needs to be injected exactly at this location, right after the `self.pubsub.publish` block, to successfully push notifications for incoming chat messages.

미구현 상태, 해당 부분을 구현함

11. 이더리움 테스트용 클라이언트 만들기 (다른 레포에 등록함)

12. 프론트엔드 css 디자인

[Role & Persona]

You are an Awwwards-winning Lead UI/UX Designer and Senior Frontend Engineer. You specialize in transforming functional interfaces into visually stunning, highly polished, and modern digital experiences.

[Strict Guardrails (CSS/UI Only)]

- **Boundary:** You must operate STRICTLY within the frontend (`crates/frontend/`).
- **No Logic Changes:** Do NOT modify backend code, API calls, state management logic, or core business rules.
- **Scope:** Your modifications are strictly limited to CSS files, style attributes, utility classes (e.g., Tailwind), and minor HTML/DOM restructuring required purely for styling and layout purposes.

[Aesthetic Directives & Design System (The "Beautiful" Standard)]

To ensure the site looks beautiful, professional, and cohesive, enforce the following UI/UX principles across the application:

1. **Visual Hierarchy & Whitespace:** Use generous, consistent padding and margins (e.g., 4-point or 8-point grid). Ensure clear spacing between distinct sections to let the UI "breathe".

2. **Typography:** Establish a clear font hierarchy. Use high contrast for primary text, softer tones for secondary text, and comfortable line heights (e.g., `1.5` or `1.6` for body text).
3. **Color Palette & Depth:** Apply a cohesive, modern color palette. Use subtle, soft box-shadows (`shadow-sm` to `shadow-lg`) and borders to create depth and separate surface layers (e.g., cards floating on a slightly darker background).
4. **Micro-interactions:** Add smooth CSS transitions (`transition-all duration-200 ease-in-out`) on interactive elements (hover states on buttons, cards, and links, as well as focus rings for accessibility).
5. **Responsiveness:** Ensure all layouts adapt gracefully to mobile, tablet, and desktop using modern Flexbox and CSS Grid techniques.

[Cognitive Architecture (Plan-and-Solve & ReAct)]

Do not blindly apply styles. Follow this systematic approach:

Phase 1: Thought (Design System Planning)

- Analyze the current UI components in `crates/frontend/`.
- Propose a consistent Design System (Primary colors, border-radius standards, spacing scale, and shadow types) that fits the product's identity.

Phase 2: Action (Execution)

- Apply the defined design system to the global stylesheets and individual components (e.g., inputs, buttons, cards, navbars).

Phase 3: Observation & Reflexion (Visual Audit)

- Mentally review the applied styles: "Is the contrast ratio readable? Are the hover states distinct? Does the layout break on small screens?"
- If any UI element feels clunky or outdated, refine it before finalizing.

Execution:

Begin immediately with **Phase 1**. Output your aesthetic strategy and target components, then execute the styling changes.

13. 블록체인 부분 수정

[Role & Persona]

You are a Senior Web3 Backend Architect and Integration Expert. You specialize in safely porting external reference implementations into existing Rust (Axum) codebases and managing dependency injection.

[Task & Problem Context]

We need to replace our backend's legacy Bitcoin Cash (BCH) logic with Ethereum (EVM) logic.

I am providing you with a fully functional, verified `BlockchainManager` implementation written in `ethers-rs` (extracted from another working client).

Your task is to **port** this provided reference code into our current

backend**, wire it up to the Axum state, ensure compatibility with our local Anvil Docker container, and completely eradicate the old BCH configurations.

```

**[Provided Reference Code (Blueprint)]**
// (Agent, assume the user will provide the BlockchainManager code block here)
// [USER: Insert the ethers-rs code block here]

```

```

use ethers::prelude::*;
use eyre::Result;
use rand::thread_rng;
use serde::{Deserialize, Serialize};

```

```

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct UserAccount {
    pub alias: String,
    pub address: String,
    pub private_key: String,
    #[serde(skip)]
    pub balance: U256,
}

```

```

impl UserAccount {
    pub fn new(alias: String) -> Self {
        let wallet = LocalWallet::new(&mut thread_rng());
        Self {
            alias,
            address: format!("{:?}", wallet.address()),
            private_key: hex::encode(wallet.signer().to_bytes()),
            balance: U256::zero(),
        }
    }
}

```

```

pub struct BlockchainManager {
    provider: Provider<Http>,
    master_wallet: LocalWallet,
    chain_id: u64,
}

```

```

impl BlockchainManager {
    pub async fn new(rpc_url: &str, master_key: &str) -> Result<Self> {
        let provider = Provider:::<Http>::try_from(rpc_url)?;
        let chain_id = provider.get_chainid().await?.as_u64();
        let master_wallet = master_key.parse:::<LocalWallet>
()?).with_chain_id(chain_id);

        Ok(Self {
            provider,
            master_wallet,
            chain_id,
        })
    }
}

```

```

    }

    pub fn provider(&self) -> &Provider<Http> {
        &self.provider
    }

    pub fn chain_id(&self) -> u64 {
        self.chain_id
    }

    pub fn master_address(&self) -> Address {
        self.master_wallet.address()
    }

    pub async fn get_balance(&self, address: Address) -> Result<U256> {
        let balance = self.provider.get_balance(address, None).await?;
        Ok(balance)
    }

    pub async fn fund_account(&self, target: Address, amount: U256) ->
    Result<TxHash> {
        // Try Anvil cheatcode first
        let is_anvil = {
            let res: Result<U256, _> =
self.provider.request("anvil_nodeInfo", ()).await;
            res.is_ok() || self.chain_id == 31337 // fallback heuristic
        };

        if is_anvil {
            // Use anvil_setBalance cheatcode
            let current_balance = self.get_balance(target).await?;
            let new_balance = current_balance.saturating_add(amount);
            let hex_balance = format!("{:016x}", new_balance);
            let address_hex = format!("{:040x}", target);

            let _: () = self.provider.request("anvil_setBalance",
(address_hex, hex_balance)).await?;
            Ok(TxHash::zero()) // No real tx hash for cheatcode
        } else {
            // Real transaction from master
            let client = SignerMiddleware::new(self.provider.clone(),
self.master_wallet.clone());
            let tx = TransactionRequest::new()
                .to(target)
                .value(amount);

            let pending_tx = client.send_transaction(tx, None).await?;
            let receipt = pending_tx.await?.ok_or_else(|| eyre::eyre!("Tx
dropped"))?;
            Ok(receipt.transaction_hash)
        }
    }

    pub async fn send_transaction(&self, from_key: &str, target: Address,

```

```

amount: U256) -> Result<TxHash> {
    let wallet = from_key.parse::<LocalWallet>
    ()?.with_chain_id(self.chain_id);
    let client = SignerMiddleware::new(self.provider.clone(), wallet);
    let tx = TransactionRequest::new()
        .to(target)
        .value(amount);

    let pending_tx = client.send_transaction(tx, None).await?;
    let receipt = pending_tx.await?.ok_or_else(|| eyre::eyre!("Tx
dropped"))?;
    Ok(receipt.transaction_hash)
}
}

```

[Cognitive Architecture: Plan-and-Solve & Safe Porting]

You are operating in Read-Write Mode. Execute the following sub-goals sequentially to implement an end-to-end integration:

Sub-goal 1: Dependency Injection & File Creation (The Porting)

Hypothesis: Since this is external code, the target backend might be missing the ethers or eyre dependencies in Cargo.toml, and needs a dedicated file for this manager.

Directive:

Add necessary dependencies (ethers, eyre, hex) to crates/backend/Cargo.toml.

Create a new module (e.g., crates/backend/src/utils/blockchain.rs or similar) and carefully port the provided BlockchainManager code into it. Export it properly in mod.rs.

Sub-goal 2: Legacy Purge & Env Alignment

Hypothesis: The current backend expects BCH_API_URL. This needs to be replaced to feed the new BlockchainManager.

Directive: Search for BCH_API_URL. Rename it to ETH_RPC_URL. Ensure the default points to our Anvil container (http://127.0.0.1:8545 or http://escrow-anvil:8545). Add MASTER_WALLET_KEY to the env configuration (using Anvil's default 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80).

Sub-goal 3: Axum State Injection in main.rs

Hypothesis: The new BlockchainManager must be instantiated at startup and shared safely across threads to be used by Axum handlers.

Directive: In crates/backend/src/main.rs, initialize the BlockchainManager. Wrap it in Arc<BlockchainManager> and inject it into the Axum AppState (or Router State). Ensure AppState implements Clone.

[Execution (ReAct Loop & Architecture Reflexion)]

For each Sub-goal, adhere strictly to this loop:

Thought: Explain your exact plan. Where will you create the new file? How will you update Cargo.toml? How will you structure the Axum AppState?

Action: Execute the tool to read, modify, and create the necessary files.

Observation & Reflexion (Mandatory Verification):

Reflexion SG1: "Did I add the ethers crate correctly? Are there any missing imports in the new file?"

Reflexion SG3: "Did I wrap the manager in an Arc? Does the compiler throw an extractor error?"

Run `cargo check --manifest-path crates/backend/Cargo.toml` after completing all steps to guarantee zero compiler errors.

Begin immediately with Sub-goal 1. Output your Thought and Action to port the reference code.

14. 에스크로 수정

[Role & Task]

You are a Senior Backend Debugging Agent specializing in Rust (Axum) and Web3 (EVM) architectures. Your task is to perform a Root Cause Analysis (RCA) for a `500 Internal Server Error` occurring when a client attempts to create an escrow.

[Strict Guardrails (Read-Only Mode)]

- **ZERO MODIFICATION POLICY:** You are operating in a strictly READ-ONLY environment.
- You must deeply analyze the backend codebase (`crates/backend/`) to trace the error stack. **Under no circumstances** are you allowed to modify, refactor, or output any rewritten code. Your final output must be purely a diagnostic RCA Report.

[Context & Observation Data]

- **Endpoint:** `POST http://localhost:8080/v1/escrow`
- **Error:** `500 Internal Server Error`
- **Hypothesis Boundary:** A 500 error implies the route matched successfully, but the server panicked or returned an unhandled error domain during execution. Focus strictly on runtime panics, unhandled `Result::Err` bubbling, EVM RPC failures, or Database constraint violations.

[Cognitive Architecture & Attention Mechanism]

Enter the **ReAct (Reason + Act)** loop. Focus your investigation sequentially on these 3 critical Attention Areas:

****Attention 1: Handler & Error Mapping (The Entry Point)****

- ****Target:**** `crates/backend/src/handlers/escrow\_handler.rs` and global error handlers (`error.rs`).
- ****Check:**** Inspect the `POST` handler for escrow creation. Is it extracting a state that doesn't exist (e.g., missing `Extension` or `State`) causing an immediate panic? Trace how errors from the service layer are mapped to `500 Internal Server Error`.

****Attention 2: Service Layer & EVM Interaction (The Business Logic)****

- ****Target:**** `crates/backend/src/service/eth\_escrow\_service.rs` (or similar service).
- ****Check:**** Look for any `.unwrap()` or `.expect()` calls that might panic. Does the service attempt to call a smart contract or the EVM node (Anvil) via `ethers-rs`? If the RPC node is down, unreachable, or a transaction reverts, does it bubble up as a 500 error?

****Attention 3: Database Persistence (The Data Layer)****

- ****Target:**** Database insertion logic for the Escrow entity (`create` operations).
- ****Check:**** Is the application trying to insert a record that violates a DB constraint? (e.g., a missing foreign key for `product\_id` or `buyer\_id`, or a unique constraint violation).

****[Execution (ReAct Loop Initialization)]****

Start the ReAct loop to generate the final RCA report:

- ****Thought:**** Explain your strategy for tracing the request lifecycle from the Axum handler down to the EVM and Database. Which exact files will you read first to find the stack trace of this 500 error?
- ****Action:**** Execute the tool to read the backend escrow handler, service, and error definition files.

(Reminder: Output purely analytical Thoughts and Actions until you have isolated the exact line or logic causing the 500 error. Conclude with a comprehensive RCA Report. DO NOT output fix codes.)

보고서를 바탕으로 예외처리를 단단하게 만드는 과정 진행

****[Mode Switch & Role Update]****

Excellent diagnostic work. The RCA report is perfectly accurate. You have successfully identified that "Error Masking" is causing transient, business, and DB errors to bubble up as a generic `500 Internal Server Error`.

You are now transitioning from a Read-Only Diagnostic Agent to an ****Autonomous Backend Architecture Agent (Read-Write Mode)**** specializing in Rust error propagation and API design.

****[Task & Sub-goal Breakdown (Plan-and-Solve)]****

Your overarching objective is to unmask the internal errors, implement granular error handling, and ensure the API returns semantically correct HTTP status codes. Execute these sub-goals sequentially:

```

**Sub-goal 1: Expand Domain Errors (Traits / Error Enum)**
- **Target:** `crates/backend/src/service/traits.rs` (or where
`EscrowServiceError` is defined) and `crates/backend/src/utls/error.rs`.
- **Directive:** Expand the `EscrowServiceError` enum. Add new specific
variants: `InvalidWalletAddress`, `ContractRevert(String)`, and
`RpcError(String)`. Ensure these are properly mapped in Axum's
`IntoResponse` (or `AppError` mapping) so that:
  - `ProductNotFound` & `TradeNotFound` -> `404 Not Found`
  - `InsufficientBalance`, `InvalidWalletAddress`, `ContractRevert` -> `400
Bad Request`
  - `RpcError` -> `502 Bad Gateway`

**Sub-goal 2: Service Layer Refactoring (EVM & DB Checks)**
- **Target:** `crates/backend/src/service/eth_escrow_service.rs`
- **Directive:**
  1. **Pre-flight Check:** Before calling the EVM contract, validate the
`seller_address` and `buyer_address`. If parsing fails, return
`EscrowServiceError::InvalidWalletAddress` instead of `Internal`.
  2. **Refine `.map_err():` Differentiate `ethers-rs` errors. Map
network/timeout errors to `RpcError` and smart contract execution failures
(out of gas, requirements not met) to `ContractRevert`. Leave true database
panic constraints as `Internal`.

**Sub-goal 3: Handler Mapping Alignment (API Layer)**
- **Target:** `crates/backend/src/handlers/escrow_handler.rs`
- **Directive:** Update the `match` block inside `initiate_escrow`.
Remove the lazy `_ => AppError::Internal` wildcard if possible, or
explicitly catch the newly defined domain errors (like `ProductNotFound`,
`InsufficientBalance`) and map them to their corresponding `AppError`
variants so the correct HTTP status code is returned to the frontend.

**[Execution (ReAct Loop & Cross-Boundary Reflexion)]**
For **each Sub-goal**, adhere strictly to this ReAct loop:

- **Thought:** State your exact error refactoring plan. How will you modify
the `EscrowServiceError` enum without breaking the `escrow_dispatcher.rs`
which might also use it?
- **Action:** Execute the tool to modify the handler, service, and error
definition files.
- **Observation & Reflexion (Mandatory Compiler Check):**
  - *Reflexion:* "Did adding new variants to `EscrowServiceError` break the
legacy `bch_escrow_service` or `escrow_dispatcher` match blocks? Does
Axum's `IntoResponse` still compile?"
  - Immediately run `cargo check --manifest-path
crates/backend/Cargo.toml`. Self-correct any missing `match` arms before
declaring the task complete.

Begin immediately with **Sub-goal 1**. Output your Thought and Action to
expand the error definitions.

```


****[Role & Task]****

You are a Senior Backend Debugging Agent specializing in Domain-Driven Design (DDD), Access Control (Authorization), and Idempotent API design. Your task is to perform a Root Cause Analysis (RCA) to verify if two specific logical flaws exist in the Escrow system's codebase.

****[Strict Guardrails (Read-Only Mode)]****

- ****ZERO MODIFICATION POLICY:**** You are operating in a strictly READ-ONLY environment.
- Do NOT rewrite or fix any code. Your sole objective is to audit the backend business logic and confirm exactly where and why these logical bugs occur.

****[Context & Observation Data]****

The escrow system is currently exhibiting two critical business logic failures:

1. ****Seller Lockout:**** The seller of a product is completely unable to access or view the escrow associated with their own product.
2. ****Buyer Single-Access Bounce:**** The buyer can initiate/access the escrow the first time. On the second attempt, instead of fetching the existing escrow, the system rejects the buyer (or crashes/bounces).

****[Cognitive Architecture & Directed Exploration (ReAct Loop)]****

Enter the ****ReAct (Reason + Act)**** loop. Investigate the codebase by focusing on these 3 critical Attention Areas:

****Attention 1: Access Control Validation (The Seller Lockout)****

- ****Target:**** `crates/backend/src/handlers/escrow\_handler.rs` and `crates/backend/src/service/eth\_escrow\_service.rs` (specifically fetch/get operations).
- ****Hypothesis:**** The backend query or authorization block strictly enforces `if escrow.buyer\_uid != user.uid { return Forbidden }`. It entirely forgets to check `|| product.seller\_uid == user.uid`, effectively locking the seller out of their own trade.
- ****Check:**** Audit the validation logic where an escrow is retrieved by its ID. Look for missing ownership checks regarding the seller.

****Attention 2: API Idempotency & State Flow (The Buyer Bounce)****

- ****Target:**** Escrow initialization logic (`create` or `initiate` operations).
- ****Hypothesis:**** When the buyer enters the product page a second time, the frontend likely calls `POST /escrow` again. The backend lacks an idempotency check (e.g., "Does an active escrow already exist for this buyer and product?"). Instead of gracefully returning the existing escrow, it attempts to insert a duplicate, triggering a Database Unique Constraint Violation or a state panic, causing the "bounce".
- ****Check:**** Verify if `initiate\_escrow` has a fallback to fetch an existing active escrow before attempting to create a new one.

****Attention 3: Database Schema & Unique Constraints****

- ****Target:**** Escrow Entity definitions (`crates/backend/src/db/entity/escrow.rs`).

- ****Check:**** Are there unique constraints on `(product\_uid, buyer\_uid)`? If so, confirm how the service layer handles `DbErr::RecordNotInserted` or unique key violations.

****[Execution (ReAct Loop Initialization)]****

Start the ReAct loop to generate the final RCA report:

- ****Thought:**** Explain your strategy to trace the Escrow Fetch and Create lifecycles. Which specific functions will you inspect to prove or disprove the Seller Lockout and Buyer Bounce hypotheses?
- ****Action:**** Execute the tool to read the escrow handler, service logic, and database entities.

(Reminder: Output purely analytical Thoughts and Actions until you have isolated the exact code blocks causing these domain flaws. Conclude with a comprehensive RCA Report. DO NOT output fix codes.)

에스크로 비즈니스 도메인 버그 수정

****[Mode Switch & Role Update]****

Excellent diagnostic work. The RCA report perfectly isolated the issues: the Buyer Bounce is caused by a lack of API idempotency during state validation, and the Seller Lockout is not a backend auth issue, meaning it must be a Frontend UI/Routing blocker.

You are now transitioning from a Read-Only Diagnostic Agent to an ****Autonomous Full-Stack Engineering Agent (Read-Write Mode)**** specializing in Idempotent API design and Frontend State logic.

****[Task & Sub-goal Breakdown (Plan-and-Solve)]****

Your objective is to fix the buyer bounce by injecting an idempotency fallback in the backend, and to fix the seller lockout by removing the frontend UI restriction. Execute these sub-goals sequentially:

****Sub-goal 1: Implement Backend Idempotency (Buyer Bounce Fix)****

- ****Target:**** `crates/backend/src/service/eth\_escrow\_service.rs` (and/or `escrow\_service.rs`), specifically the `initiate` function.
- ****Directive:**** Before checking `if p.status != "on\_sale"`, query the database for an existing active `escrow\_trade` where `product\_id == req.product\_id` AND `buyer\_id == user\_id`.
 - If a valid existing trade is found, ****return it early (Idempotency)**** instead of throwing an `InvalidState` error.
 - If no trade exists, proceed with the normal creation logic.

****Sub-goal 2: Unblock Seller UI Access (Seller Lockout Fix)****

- ****Target:**** `crates/frontend/src/pages/product\_detail.rs` (or equivalent product/escrow UI components).
- ****Directive:**** Since the backend allows sellers to fetch the trade, the frontend must be conditionally hiding the "View Escrow" button or routing incorrectly if the current user is the seller.
 - Inspect the frontend logic. Look for blocks like `if is\_buyer { show\_escrow() }`.
 - Modify the logic so that ****both the buyer and the seller**** can see the button and navigate to the active escrow room. Ensure the correct

`escrow\_uid` is passed to the route.

[Execution (ReAct Loop & Cross-Boundary Reflexion)]

For **each Sub-goal**, adhere strictly to this ReAct loop:

- **Thought:** State your logic. For SG1, how will you construct the SeaORM query to find the existing trade? For SG2, which frontend files will you search to find the UI blocker?
- **Action:** Execute the tool to modify the backend service and the frontend UI components.
- **Observation & Reflexion (Mandatory Verification):**
 - **Reflexion SG1:** "If the buyer clicks 'Buy' twice, does the backend now return `Ok(existing\_trade)` instead of `Err(InvalidState)`?"
 - **Reflexion SG2:** "Is the seller now able to click the escrow button on their own product page without being blocked by the UI?"
 - Immediately run `cargo check --manifest-path crates/backend/Cargo.toml` and `cargo check --manifest-path crates/frontend/Cargo.toml` to ensure your database queries and UI changes compile correctly.

Begin immediately with **Sub-goal 1**. Output your Thought and Action to implement idempotency in the backend.

15. wallet 잔액이 evm, db쪽에 둘다 가지고 있으며 생기는 문제점을 수정

evm을 조회하며 동적으로 유지하는 방향으로 수정함

<role>

You are a Principal Web3 Rust Backend Architect and SeaORM & EVM Database Architecture Specialist.

You specialize in ensuring state consistency between on-chain data and off-chain databases, adhering strictly to the Single Source of Truth (SSOT) principle, and engineering fault-tolerant Rust backend systems.

</role>

<problem_statement>

Currently, the `wallets` entity/table contains a `balance: Decimal` column. This leads to a critical logical bug where the database balance remains unchanged after on-chain transactions, causing state desynchronization between off-chain database records and actual on-chain wallet balances.

In the EVM ecosystem, the **on-chain state (Blockchain RPC Node)** is the sole Single Source of Truth for wallet balances. Storing balances redundantly in the database must be completely deprecated, and the system must be refactored to query balances dynamically via RPC calls.

</problem_statement>

<entity_to_refactor>

// [Target File] crates/backend/src/entity/wallets.rs

```
#[sea_orm(table_name = "wallets")]
pub struct Model {
    #[sea_orm(primary_key)]
    pub id: i64,
    pub user_id: i64,           // FK → users.id
    pub eth_address: String,    // EVM Public Address
    pub balance: Decimal,      // [TO BE REMOVED] Root cause of
balance bugs due to redundant DB storage
    pub created_at: DateTimeWithTimeZone,
    pub updated_at: DateTimeWithTimeZone,
}
</entity_to_refactor>
```

<strict_constraints>

1. ****Single Source of Truth (SSOT):**** Remove the `balance` field from the `wallets` table/entity. User balance queries MUST be fetched dynamically via on-chain RPC calls (`get\_balance`) using `ethers-rs` Provider.
 2. ****Safe Available Balance Calculation:****
`Available Withdrawal Funds = (On-chain Real Balance via RPC) - (Sum of Active Escrow Locked Amounts in DB)`
 3. ****Relationship & Integrity Audit:**** Analyze the relationships between `wallets`, `escrow`, and `wallet\_transaction`. Refactor the architecture so off-chain DB records serve solely as transaction logs (history) and escrow state tracking.
 4. ****Rust Compilation Guarantee:**** Update all existing services (`wallet\_service.rs`, `eth\_escrow\_service.rs`, etc.) that previously referenced `wallets.balance`. Upon task completion, run `cargo check --manifest-path crates/backend/Cargo.toml` to ensure zero compilation errors.
- </strict_constraints>

<execution_plan>

[Plan-and-Solve: 4-Phase Sequential Roadmap]

- ****Phase 1: Entity & Transaction Relationship Mapping****
 - Analyze relationships between `wallets`, `escrow`, and `wallet\_transaction` entities and define the Single Responsibility for each table.
 - Verify that `wallet\_transaction` functions strictly as an off-chain history log of on-chain transactions.
- ****Phase 2: DB Model & Migration Refactoring****
 - Remove the `balance` column from the SeaORM `wallets` entity.
 - Apply the migration script or update the model accordingly.
- ****Phase 3: RPC-based Dynamic Balance Service Implementation****
 - Inject the `ethers-rs` `Provider` into `wallet\_service.rs` to implement `get\_onchain\_balance` for real-time on-chain queries based on `eth\_address`.
 - Safely handle conversions between Wei (`U256`) units and human-readable Decimal/Ether representations.
- ****Phase 4: Dependent Services Refactoring (`eth\_escrow\_service.rs` & Withdrawal Guard)****
 - Remove all legacy DB update calls mutating `wallet.balance` during

escrow settlements, payouts, or withdrawals.

- Refactor the Withdrawal Guard to enforce: `(On-chain RPC Balance - Active Escrow Locked Amount) >= Requested Amount`.

</execution_plan>

<react_reflexion_protocol>

Adhere strictly to the following ReAct & Reflexion self-verification cycle at each phase:

- **Thought:**

- Analyze current code flaws and design the refactored file/code structure for the current Phase.

- Identify all downstream service side-effects resulting from removing `wallets.balance`.

- **Action:**

- Execute code editing tools to modify entities, refactor services, and integrate RPC logic.

- **Observation & Reflexion (Mandatory Verification):**

- [Reflexion 1]: "Are there any lingering DB `UPDATE` queries attempting to mutate wallet balances in any service logic?"

- [Reflexion 2]: "Is RPC network communication failure properly handled using `Result<T, E>` without panicking?"

- [Reflexion 3]: "Does `cargo check` compile successfully without errors?"

</react_reflexion_protocol>

<instruction>

Begin immediately with **Phase 1 (Entity & Transaction Relationship Mapping)**.

Output your **Thought** and analysis on how to redefine relationships between `wallets`, `escrow`, and `wallet\_transaction`, and how to eliminate the `balance` column safely.

</instruction>

또한 기존에 존재했던 취약점 수정

16. 백엔드의 변경에 맞게 프론트엔드 변경

<role>

You are a Principal Web3 Frontend Architect and DApp Integration Specialist.

You specialize in smart contract event/transaction handling, Web3 wallet integrations (MetaMask, WalletConnect), API state machine synchronization, and building type-safe asynchronous Web3 workflows.

</role>

<context>

The backend escrow smart contract integration engine refactoring is now complete.

The backend no longer initiates deposit transactions via a system hot wallet; instead, it enforces a **decentralized deposit workflow** where buyers broadcast deposits directly from their Web3 wallets.

[Backend Walkthrough Summary & Frontend Requirements]

1. **Updated `initiate` Response:** Calling `POST /v1/escrow` now returns `tx\_params` (containing hex `to`, `value`, `data`) and an escrow status of `pending\_deposit`.
2. **Web3 Wallet Signing & Broadcasting:** The frontend must extract `tx\_params` and pass it to the user's wallet via `window.ethereum.request({ method: "eth\_sendTransaction", params: [txParams] })` (or equivalent ethers/viem calls).
3. **On-Chain Deposit Verification (`verify-deposit`):** Upon broadcasting the transaction, the frontend must capture the returned `tx\_hash` and dispatch a request to `POST /v1/escrow/:trade\_uid/verify-deposit` so the backend can verify the transaction on-chain and update the status to `deposited`.

</context>

<strict_constraints>

1. **Strict 3-Step Deposit Pipeline:** Enforce the explicit pipeline: `[1. Call Backend initiate] -> [2. Sign/Broadcast via Web3 Wallet & Capture tx\_hash] -> [3. Call Backend verify-deposit]`.
2. **Transaction Parameter Precision:** Ensure `tx\_params` (`to`, `value`, `data`) are passed to the Web3 provider without formatting loss or hex encoding corruption.
3. **Comprehensive Error & Rejection Handling:** Implement resilient exception traps for scenarios such as user rejection (`User rejected transaction`), insufficient gas, RPC timeouts, and transaction failures to prevent breaking the UI state.
4. **Build Validation:** Upon task completion, run frontend compilation/typecheck commands (e.g., `cargo check` or `npm run build` / `tsc`) to guarantee zero build errors.

</strict_constraints>

<execution_plan>

[Plan-and-Solve: 3-Phase Frontend Refactoring Roadmap]

- **Phase 1: Escrow API Client & Type Definitions Update**
 - **Target:** API service and type definition files inside `crates/frontend`.
 - Update `initiate` API response models to include `tx\_params` (`to`, `value`, `data`) and the `pending\_deposit` status state.
 - Implement API client functions and TypeScript/Rust-WASM interfaces for `POST /v1/escrow/:trade\_uid/verify-deposit`.
- **Phase 2: Web3 Provider & Deposit Pipeline Integration**
 - **Target:** Wallet provider module and escrow checkout components/services inside `crates/frontend`.
 - Wire the payment flow so that after calling `initiate`, the returned `tx\_params` are immediately passed to the Web3 Provider (`window.ethereum` / `ethers.js` / `viem`) to broadcast the transaction.

```

- Pipeline the handler to capture the resulting `tx_hash` and trigger
`POST /v1/escrow/:trade_uid/verify-deposit`.

- Phase 3: UI State Machine & UX Error Handling Refactoring
  - Target: Escrow checkout UI components and state handlers.
  - Implement state visualizers for `pending_deposit` ("Awaiting Wallet
Signature" -> "Verifying On-Chain Deposit").
  - Add toast notifications/modal states for user transaction rejections
and network error recovery.
</execution_plan>

<react_reflexion_protocol>
Adhere strictly to the following ReAct & Reflexion self-verification cycle
at each phase:

- Thought:
  - Analyze the integration gap between backend walkthrough requirements
and current frontend code.
  - Design the data flow for Web3 wallet signing and the two-step API
verification pipeline.
- Action:
  - Execute code modification tools to update frontend API clients, wallet
providers, and UI components.
- Observation & Reflexion (Mandatory Verification):
  - [Reflexion 1]: "Are `tx_params` passed to the Web3 provider without hex
encoding corruption?"
  - [Reflexion 2]: "Does the pipeline automatically trigger `verify-
deposit` after capturing the `tx_hash`?"
  - [Reflexion 3]: "Does the frontend compile cleanly with zero type/build
errors?"
</react_reflexion_protocol>

<instruction>
Begin immediately with Phase 1 (Escrow API Client & Type Definitions
Update).
Output your initial Thought and Action detailing the files to
modify in `crates/frontend` and the updated API type definitions.
</instruction>

```

17, BCH 관련 잔재 완전 삭제

Remove BCH Support from Backend

This implementation plan outlines the complete removal of all legacy Bitcoin Cash (BCH) support from the backend application, preserving only the recently verified ETH wallet and escrow functionality.

Goal

Remove all BCH-related business logic, abstractions, dependencies, and infrastructure, reducing the codebase to an ETH-only model while maintaining database compatibility.

Phase 0: Findings (BCH Dependency Inventory)

A repository-wide analysis revealed the following BCH footprints:

1. Active BCH Business Logic

WalletDomain: Contains conditional branches (if currency == "ETH") and dedicated BCH balance calculation loops.

WalletService: Legacy withdrawal logic for req.amount_bch.

EscrowDispatcherService: Routes requests between bch_service and eth_service.

EscrowService: The legacy BCH-specific escrow implementation.

Tests in wallet_domain_test.rs and escrow_service_test.rs.

2. BCH-Specific Infrastructure & Ports

bch_network_adapter.rs and bch_network_port.rs: Electrum HTTP gateway clients.

Environment variables: BCH_API_URL (in .env and bch_network_adapter.rs).

EscrowDispatcherService itself (only needed to support multiple currencies).

3. BCH-Specific API Contracts & DTOs

Currency enum in common_dto.rs contains BCH.

TransactionDto contains amount_bch and public_address (CashAddr).

AdminDto contains total_accumulated_bch.

BchAmount struct in types.rs.

4. BCH-Specific Database Schema / History

products.currency and escrow_trades.currency store currency strings (default 'ETH').

platform_stats.total_fee_collected was mapped to total_accumulated_bch in DTOs.

Phase 1: Target ETH-Only Architecture

The target architecture simplifies the domain by eliminating multi-currency abstractions where they no longer serve a purpose:

Wallet Domain: calculate_balance_info will operate exclusively on ETH logic (on-chain balance minus pending_deposit lock). Off-chain BCH tracking logic will be deleted entirely.

Escrow: EscrowDispatcherService and the legacy EscrowService will be deleted. EthEscrowService will take over as the sole implementation of EscrowServiceTrait.

Database: We will NOT drop the currency columns in products or escrow_trades yet to avoid breaking historical records or requiring destructive migrations. However, backend entity mappers will default these or ignore them.

WARNING

The Currency enum in common_dto.rs will be reduced to only ETH, or completely removed if we change the API to no longer require the user to specify currency (since ETH is the only one). I propose keeping Currency::ETH and dropping BCH to maintain API signature stability for clients for now, unless you prefer complete removal of the currency fields.

IMPORTANT

AdminDto::total_accumulated_bch maps directly to the database column platform_stats.total_fee_collected. Should we rename this DTO field to total_accumulated_fees (breaking API change for frontend), or keep the _bch suffix to avoid frontend breakage? I propose renaming it to total_accumulated_eth or total_accumulated_fees and making the same change in the frontend.

Proposed Changes

[Core Wallet & Escrow Services]

[MODIFY]

wallet_domain.rs

Delete offchain_bch_balance, wallet_locked_amount_bch, available_bch_balance.

Delete the BCH-specific loop over wallet_transactions in get_balance_info. Simplify calculate_balance_info to only calculate ETH locks (pending_deposit = wallet lock; all others = display lock).

[MODIFY]

wallet_service.rs

Remove req.amount_bch references.

[DELETE]

escrow_dispatcher.rs

Delete the dispatcher since there's only one implementation.

[DELETE]

escrow_service.rs

Delete the legacy BCH implementation.

[MODIFY]

main.rs

Remove bch_escrow_service and EscrowDispatcherService DI wiring. Wire EthEscrowService directly as the EscrowServiceTrait.

[Infrastructure & Ports]

[DELETE]

bch_network_port.rs

Delete entirely.

[DELETE]

bch_network_adapter.rs

Delete entirely.

[MODIFY] [mod.rs files in ports and infra]

Remove pub mod bch_network_port and pub mod bch_network_adapter.

[MODIFY] [.env]

Remove BCH_API_URL.

[DTOs and Database Entities]

[MODIFY]

common_dto.rs

Remove BCH from the Currency enum.

Add an UNSUPPORTED variant with #[serde(other)] to act as a read-only legacy representation for historical records, strictly adhering to the rule to NOT default unknown currencies to ETH.

[MODIFY]

transaction_dto.rs

Remove amount_bch.

PRESERVE public_address. Code review confirms it holds the user's ETH address and is actively consumed by the frontend for display.

```

[MODIFY]
types.rs
Remove BchAmount.
[MODIFY] [db/entity/escrow_trade.rs & product.rs]
Update try_from mappings so that _ => Currency::UNSUPPORTED. Any legacy BCH
record will be explicitly marked as unsupported rather than silently
migrating to ETH.
[MODIFY] [admin_dto.rs / platform_stats.rs / admin.rs (frontend)]
Rename total_accumulated_bch to total_accumulated_fees across both backend
and frontend. The unit in the frontend is already explicitly "ETH", so this
preserves semantic correctness.
[Tests]
[MODIFY]
wallet_domain_test.rs
Remove BCH-specific scenarios (test_bch_escrow_lifecycle,
test_mixed_currencies).
Keep and expand ETH-specific regression tests.
[DELETE]
escrow_service_test.rs
Delete this legacy BCH test suite.
Verification Plan
Automated Tests
Run cargo check -p backend -p frontend to ensure no dead BCH references
block compilation.
Run cargo test -p backend to ensure the ETH invariants remain intact
without BCH complexity.
Run cargo clippy -p backend -- -D warnings to catch unused imports
resulting from BCH removal.
Manual Verification
Perform a final repository-wide grep -i -E 'bch|bitcoin cash' to ensure
absolute eradication of BCH business logic.

```

-> 송금 기능 구현 포기

18. 타유저 열람 구현

Public User Profile Implementation Plan

Goal Description

Implement a public user profile viewing feature allowing users to view a seller's registration date, public user ID, and active products.

Proposed Changes

Shared DTOs

Add PublicUserProfileRes to crates/shared/src/dto/user_dto.rs with user_uid, display_name, bio, reliability_index, and joined_at.

Add owner_uid to ProductSearchQuery in crates/shared/src/dto/product_dto.rs.

Backend: User Service & Handler

Add get_public_profile method to UserServiceTrait and UserService in crates/backend/src/service/user_service.rs. It will map user::Model to

```

PublicUserProfileRes.
Add get_public_profile handler in
crates/backend/src/handlers/user_handler.rs handling GET /users/:user_uid.
Register the new route in crates/backend/src/router/mod.rs (needs to be
verified where routes are defined).
Backend: Product Service
Update list_products in ProductService
(crates/backend/src/service/product_service.rs) to filter by owner_uid if
present in ProductSearchQuery. It will decode the owner_uid to seller_id.
Frontend: Components & Pages
Add fetch_public_profile to crates/frontend/src/models/user_model.rs (or
similar).
Create crates/frontend/src/pages/public_profile.rs using existing
PageContainer, AppButton, ItemSummaryRes (if they have a ProductCard
component, reuse it).
Register /users/:user_uid route in crates/frontend/src/router.rs.
Update crates/frontend/src/pages/product_detail.rs to wrap the Seller Info
in a clickable link A pointing to /users/{item.owner_uid}.
Verification Plan
Automated Tests
cargo check -p backend -p frontend
cargo clippy -p backend -- -D warnings
Manual Verification
Will test the flow: Product Detail -> Click Seller -> Profile -> Click
Product -> Product Detail.

```

19. 보안 기능 점검 및 구현

보안 감사 및 구현 계획

****사용자 검토 필요****

아래 보안 감사 결과를 검토해 주세요. 이번 감사에서는 서버 측 검증이 누락된 일부 영역과 웹소켓 (WebSocket) 채팅 메시지 인가가 불완전한 부분이 확인되었습니다. 승인해 주시면 지정된 파일에 수정 사항을 적용하겠습니다.

Phase 0: 아키텍처 탐색

인증 (Authentication)

- * 애플리케이션은 `Authorization` 헤더 또는 `token=` 쿼리 파라미터를 통해 전달되는 JWT 기반 인증 (Bearer 토큰)을 사용합니다.
- * 세션은 상태를 유지하지 않는 스테이트리스 (Stateless) 방식이며, 수명이 짧습니다 (24시간 만료).
- * 브라우저가 교차 출처 (Cross-Origin) 요청에 Bearer 토큰을 자동으로 첨부하지 않으므로, 기존

의 CSRF 보호 방식(Anti-CSRF 토큰 등)은 해당 없음(N/A)입니다.

비밀번호 보안 (Password Security)

- * 데이터베이스에 저장하기 전 Argon2id를 사용해 비밀번호를 올바르게 해시화하고 있습니다 (`crates/backend/src/service/auth\_service.rs`).
- * 비밀번호는 로그에 기록되지 않으며, 사용자별로 솔트(Salt)가 안전하게 생성됩니다.

세션 / 토큰 보안 (Session / Token Security)

- * 토큰에는 만료 시간(`exp`)이 포함되어 있으며, 휴면/정지된 계정은 `authenticate` 미들웨어에서 적극적으로 차단됩니다.
- * 리프레시 토큰이나 세션 무효화는 짧은 만료 시간과 DB 상태 확인에 의존합니다.
- * 앱이 인증에 쿠키를 사용하지 않으므로 쿠키 보안은 해당 없음(N/A)입니다.

인가 (Authorization)

- * 커스텀 `authenticate` 미들웨어가 JWT 클레임을 파싱하여 요청 확장(request extensions)에 주입합니다.
- * 리소스 소유권은 백엔드에서 엄격하게 검증됩니다 (예: `product\_service.update\_product`에서 `seller\_id`를 DB 레코드와 비교 확인).
- * Admin 라우트에 대해서는 `require\_admin` 미들웨어를 통한 역할 기반 접근 제어(RBAC)가 적용되어 있습니다.

웹소켓 (WebSocket)

- * 웹소켓은 Rust 애플리케이션의 `/v1/chat/ws` 엔드포인트에서 직접 노출됩니다.
- * 연결 업그레이드 시 `token` 쿼리 파라미터를 사용하여 인증을 수행합니다.
- * 인스턴스 간 채팅 메시지를 확장(Scale)하기 위해 Redis Pub/Sub을 사용합니다.
- * Rust 라우터에 TLS 설정이 없으므로, WSS(TLS 종단)는 리버스 프록시(예: Nginx, Cloudflare)에서 처리하는 것으로 전제합니다.

신고 기능 (Reporting)

- * `report\_handler` 및 `report\_service`에서 처리합니다.
- * 사유(cause) 길이를 검증합니다.
- * DB 쿼리 및 트랜잭션을 사용해 중복 신고 및 본인 신고를 방지합니다.
- * 신고 임계값을 기반으로 한 자동 제재(휴면/차단) 기능이 포함되어 있습니다.

오류 처리 (Error Handling)

- * `AppError`는 내부 오류를 일반적인 HTTP 응답(예: `Internal`, `BadRequest`)으로 안전하게 매핑하여 스택 트레이스나 SQL 쿼리가 유출되지 않도록 합니다.
- * 상세 오류는 `tracing::error!`를 사용하여 안전하게 로그로 남깁니다.

Phase 1: 사용자 보안 감사

통제 항목	상태	증거	조치 사항
***서버 측 검증**	***미흡 (PARTIAL)**	`user_handler.rs`: `update_my_profile`에 `req.validate()` 누락	`update_my_profile` 핸들러에 `req.validate()` 추가

| ****XSS 보호**** | ****해당 없음 (N/A)**** | JSON API (HTML을 반환하지 않음) | 프론트엔드의 안전한 렌더링으로 처리됨 |

| ****CSRF**** | ****해당 없음 (N/A)**** | 쿠키가 아닌 Bearer 토큰 사용 | 없음 |

| ****비밀번호 해싱**** | ****통과 (PASS)**** | `auth\_service.rs:122` (Argon2) | 없음 |

| ****쿠키 보안**** | ****해당 없음 (N/A)**** | 쿠키 사용 안 함 | 없음 |

| ****세션 만료**** | ****통과 (PASS)**** | `auth\_service.rs:230` (1일 만료) | 없음 |

| ****로그인 실패 보호**** | ****미흡 (PARTIAL)**** | `auth\_handler.rs:login`에 최대 5회 제한이 구현되어 있으나, `router.rs`에서 `/auth/sendcode`에 대한 `email\_rate\_limit\_middleware`가 누락됨 | `/auth/sendcode`에 속도 제한(Rate Limit) 미들웨어 추가 |

| ****오류 처리**** | ****통과 (PASS)**** | `AppError` 매핑 | 없음 |

Phase 2: 상품 보안 감사

| 통제 항목 | 상태 | 증거 | 조치 사항 |

| --- | --- | --- | --- |

| ****서버 측 검증**** | ****통과 (PASS)**** | `product\_handler.rs`에서 생성 및 수정 시 `req.validate()` 호출 | 없음 |

| ****XSS 보호**** | ****해당 없음 (N/A)**** | JSON API | 없음 |

| ****인증**** | ****통과 (PASS)**** | `router.rs`에서 `/products` (POST/PUT/PATCH)를 `authenticate`로 보호 | 없음 |

| ****소유권 인가**** | ****통과 (PASS)**** | `product\_service.rs`에서 `p.seller\_id != seller\_id` 검증 | 없음 |

| ****데이터 무결성**** | ****통과 (PASS)**** | `create\_product`에서 트랜잭션 사용 | 없음 |

Phase 3: 채팅 보안 감사

| 통제 항목 | 상태 | 증거 | 조치 사항 |

| --- | --- | --- | --- |

| ****메시지 본문 검증**** | ****취약 (FAIL)**** | `chat\_handler.rs`: `send\_message`에 `req.validate()` 누락 | 핸들러에 `req.validate()` 추가 |

| ****웹소켓 인증**** | ****통과 (PASS)**** | `chat\_handler.rs`: `handle\_socket`에서 `claims.sub` 사용 | 없음 |

| ****메시지 인가**** | ****취약 (FAIL)**** | `chat\_service.rs`: `send\_message`에서 발신자가 해당 채팅방에 속해 있는지 검증하지 않음 | `chat\_service.rs`에 참여자 검증 로직 추가 |

| ****메시지 검증**** | ****통과 (PASS)**** | `sender\_uid`를 JWT 클레임에서 안전하게 가져옴 | 없음 |

| ****속도 제한 (Rate Limiting)**** | ****해당 없음 (N/A)**** | 표준 애플리케이션 레이어 | 없음 |

| ****WSS / TLS**** | ****해당 없음 (N/A)**** | 리버스 프록시를 통하는 것으로 전제 | 없음 |

Phase 4: 신고 기능 보안 감사

| 통제 항목 | 상태 | 증거 | 조치 사항 |

| --- | --- | --- | --- |

| ****입력값 검증**** | ****통과 (PASS)**** | `report\_handler.rs`에서 `req.validate()` 사용 | 없음 |

| ****인증**** | ****통과 (PASS)**** | `router.rs`에서 `/reports` 보호 | 없음 |

```

| **데이터 무결성** | **통과 (PASS)** | 트랜잭션 사용, 본인/중복 신고 방지 | 없음 |
| **감사 로깅** | **미흡 (PARTIAL)** | DB 로그 존재함 | 없음 |
| **남용 방지** | **통과 (PASS)** | `report_service.rs`:
`apply_auto_moderation` 구현 | 없음 |

---

## 수정 제안 사항

* `crates/backend/src/handlers/user_handler.rs`
* **[수정]** `update_my_profile`에 누락된 `use validator::Validate;` 및
`req.validate().map_err(...)` 추가.

* `crates/backend/src/handlers/chat_handler.rs`
* **[수정]** `send_message`에 누락된 `use validator::Validate;` 및
`req.validate().map_err(...)` 추가.

* `crates/backend/src/handlers/escrow_handler.rs`
* **[수정]** `initiate_escrow` 및 `dispute_escrow`에 누락된 `use
validator::Validate;` 및 `req.validate().map_err(...)` 추가.

* `crates/backend/src/service/chat_service.rs`
* **[수정]** 메시지를 데이터베이스에 삽입하기 전에 `sender_id`가 `room_id`의 활성 참여자인
지 검증하도록 `send_message` 업데이트.

* `crates/backend/src/router.rs`
* **[수정]** 이메일/SMS 인증 무차별 대입(Brute-force) 공격을 방지하기 위해
`/auth/sendcode` 라우트를
`crates::middleware::limit::email_rate_limit_middleware`로 감싸기.

---

## 검증 계획

### 자동화 테스트

* 코드가 정상적으로 컴파일되는지 확인하기 위해 `cargo check` 및 `cargo clippy` 실행.
* 보안 리그레션 테스트를 통과하는지 확인하기 위해 `cargo test` 실행.

```

20. 어드민 기능 점검

어드민 권한 주기만 추가함, 이외에는 더미코드